Erdős's conjecture on large totient fibers, conditional on smooth shifted primes: a formal verification

Abstract

For a positive integer n let $g(n)$ be the number of $m \geq 1$ with $\varphi(m) = n$. Erdős conjectured that for every $\varepsilon > 0$ the inequality $g(n) > n^{1-\varepsilon}$ holds for infinitely many n. The source paper derives this from a lower bound for primes p in $(2x, 5x]$ with $p - 1$ being x^{δ}-smooth, by the product-and-pigeonhole argument of Pomerance. We record a Lean 4/Mathlib verification of that derivation, with the smooth-shifted-primes bound taken as an explicit hypothesis. The verification also proves, unconditionally, that every totient fiber is finite. The quantitative upper bound $g(n) \ll_{\eta} n^{1+\eta}$ from the source's Lemma 1 is not formalized, and the proof does not need it.

Source	openai/math@fd4aeeb, family 11, OpenAIMath_011_TotientFibersFromSmoothShiftedPrimes
Lean	leanprover/lean4:v4.23.0, Mathlib 37df177aaa
Machine checks	compiles; no `sorry`; statement matches target; module build
Axioms	`propext`, `Classical.choice`, `Quot.sound`
Independent review	FAITHFUL (claude-opus-5-5)
Agents	captain claude-opus-5-5; solvers aiprover; report claude-opus-5-5
Run	`srv_20261008_212507_OpenAIMath_011_TotientFibersFromSmoothShiftedPrimes`

Contents

1 Statement **1**

2 Formalization **2**

3 Proof **3**

3.1 Overview 3

3.2 Proof of the theorem 30

4 Independent review **31**

A Complete Lean file **32**

1 Statement

Let φ be Euler's totient function. For a positive integer n put

$$g(n) = \#\{m \geq 1 : \ \varphi(m) = n\}.$$

For an integer $m > 1$, $P^{+}(m)$ denotes its largest prime factor. We say m is *y-smooth* if $P^{+}(m) \leq y$.

Hypothesis (H) (smooth shifted primes; Theorem 1.2 of the source paper, taken here as an assumption). For every fixed $0 < \delta < 1/4$ and every $\eta > 0$ there is x_0 such that for all real $x \geq x_0$,

$$\#\{p : \ p \text{ prime}, \ 2x < p \leq 5x, \ P^+(p-1) \leq x^\delta\} \ \geq \ x^{1-\eta}.$$

Theorem 1.1 (Erdős's conjecture on large totient fibers, conditional on (H))**.** *Assume (H). Then for every real $\varepsilon > 0$ there are infinitely many positive integers n with $g(n) > n^{1-\varepsilon}$.*

Along the way the source proves the following unconditional statement (Lemma 1): each fiber $\{m : \varphi(m) = n\}$ is finite, and $g(n) \ll_\eta n^{1+\eta}$ for every $\eta > 0$.

2 Formalization

The formalization uses Mathlib. From the library it takes the following objects:

- the totient φ;
- the set $\mathrm{pf}(m)$ of prime factors of $m \in \mathbb{N}$, with $\mathrm{pf}(0) = \mathrm{pf}(1) = \emptyset$;
- the p-adic valuation $v_p(m)$;
- the cardinality Nat.card of a type, which is defined to be 0 for infinite types;
- the real logarithm and the real power x^y;
- the natural floor $\lfloor y \rfloor_+$, equal to $\lfloor y \rfloor$ for $y \geq 0$ and to 0 for $y < 0$;
- the binary logarithm $\lg(N)$, the largest L with $2^L \leq N$ when $N \geq 1$, and $\lg(0) = 0$.

Natural numbers are cast to $\mathbb{R}$ silently in what follows. In $\mathbb{N}$ the expression $p - 1$ is truncated subtraction. For a prime p it is the ordinary $p - 1$.

Definition 2.1 (fiber size)**.** For $n \in \mathbb{N}$ let $g(n) = \mathrm{Nat.card}\{m \in \mathbb{N} : \ 0 < m, \ \varphi(m) = n\}$.

Lean: `totientFiberCard`

```
/-- `g(n) = #{m ≥ 1 : φ(m) = n}`. The fiber is finite (Lemma 1). If it were infinite,
`Nat.card` would return `0`, which could only make the theorem harder. -/
noncomputable def totientFiberCard (n : ℕ) : ℕ :=
  Nat.card {m : ℕ // 0 < m ∧ Nat.totient m = n}
```

If a fiber were infinite, this definition would give $g(n) = 0$. Lemma 3.1 shows that every fiber is finite, so $g(n)$ is the true count. For $n = 0$ the fiber is empty.

Definition 2.2 (smoothness)**.** For $m \in \mathbb{N}$ and $y \in \mathbb{R}$, m is *y-smooth* if every $q \in \mathrm{pf}(m)$ satisfies $q \leq y$.

Lean: `IsSmoothNat`

```
/-- `m` is `y`-smooth: every prime factor of `m` is at most `y`.
(For `m > 1` this is `P⁺(m) ≤ y`.) -/
def IsSmoothNat (m : ℕ) (y : ℝ) : Prop :=
  ∀ q ∈ m.primeFactors, (q : ℝ) ≤ y
```

For $m > 1$ this says exactly $P^+(m) \le y$. The integers 0 and 1 are y-smooth for every y. This edge case is irrelevant: (H) only matters for large x, since x_0 may be enlarged, and the proof uses it only with $x \ge 1$. There $p > 2x \ge 2$, so $p - 1 > 1$.

Definition 2.3 (Hypothesis (H))**.** *(H)* is the proposition: for all reals δ, η with $0 < \delta < 1/4$ and $\eta > 0$ there exists $x_0 \in \mathbb{R}$ such that for all real $x \ge x_0$,

$$\#\{p \in \mathbb{N} : \ p \text{ prime}, \ 2x < p \le 5x, \ p - 1 \text{ is } x^{\delta}\text{-smooth}\} \ \ge \ x^{1-\eta}.$$

Lean: `SmoothShiftedPrimesHypothesis`

```
/-- Hypothesis (H): for every `0 < δ < 1/4` and every `η > 0` there is `x₀` such that
for all real `x ≥ x₀`, `#{p prime : 2x < p ≤ 5x, P⁺(p-1) ≤ x^δ} ≥ x^(1-η)`. -/
def SmoothShiftedPrimesHypothesis : Prop :=
  ∀ δ : ℝ, 0 < δ → δ < 1 / 4 → ∀ η : ℝ, 0 < η →
    ∃ x₀ : ℝ, ∀ x : ℝ, x₀ ≤ x →
      (Nat.card {p : ℕ // p.Prime ∧ 2 * x < (p : ℝ) ∧ (p : ℝ) ≤ 5 * x ∧
          IsSmoothNat (p - 1) (x ^ δ)} : ℝ) ≥ x ^ (1 - η)
```

The counted set lies in $[0, 5x]$, so it is finite and Nat.card is its true size. This is Hypothesis (H) word for word.

The target statement is the following.

Lean: target statement `openaimath_011_totientfibersfromsmoothshiftedprimes`, fixed before the proof; `solution` is checked to have exactly this type

```
theorem openaimath_011_totientfibersfromsmoothshiftedprimes
    (hH : SmoothShiftedPrimesHypothesis) (ε : ℝ) (hε : 0 < ε) :
    Set.Infinite {n : ℕ | 0 < n ∧ ((totientFiberCard n : ℕ) : ℝ) > (n : ℝ) ^ (1 - ε)}
```

It takes (H) as a hypothesis, which stands for Theorem 1.2 of the source. It also takes a real $\varepsilon > 0$. It asserts that the set

$$\{n \in \mathbb{N} : \ 0 < n, \ g(n) > n^{1-\varepsilon}\}$$

is infinite. Since g is the true fiber size, this is exactly the source theorem. The theorem `solution` below has precisely this form.

Of the source's Lemma 1, only the finiteness of fibers is formalized (Lemma 3.1). The bound $g(n) \ll_\eta n^{1+\eta}$ is not.

3 Proof

3.1 Overview

The formal proof follows the product-and-pigeonhole argument of the source, with these steps:

1. Lemma 3.1 proves that fibers are finite. Lemma 3.2 turns this into a usable lower bound for $g(n)$.
2. Lemmas 3.3 and 3.4 give the identity $\varphi(\prod_{p \in Q} p) = \prod_{p \in Q}(p - 1)$ and the injectivity of $Q \mapsto \prod_{p \in Q} p$ (unique factorization).
3. Lemma 3.5 is the pigeonhole step. If all k-subsets of a set S of primes have their values $\prod(p - 1)$ in a finite set V, then some value n satisfies $\binom{|S|}{k} \le |V|\, g(n)$.

4. Lemmas 3.6 and 3.7 make estimate (2) of the source precise. A product of k shifted primes $p-1 \le N$ that are y-smooth is a y-smooth integer in $(0, N^k]$. There are at most $(\lg(N^k)+1)^{\lfloor y \rfloor_+}$ such integers.

5. Lemma 3.9 is the binomial inequality (1).

6. Lemma 3.10 through Lemma 3.17 carry out the asymptotic comparison of (1) and (2) with explicit constants. This replaces the source's "$X^{o(k)}$".

7. Lemmas 3.18 to 3.21 assemble the argument.

The formal proof deviates from the source in the following points.

- *Scale.* The source passes to $X = 5x$ and to primes $p \le X$. The formal proof works directly with the parameter x of (H), the primes in $(2x, 5x]$, and $N = \lfloor 5x \rfloor_+$.
- *Parameters.* It fixes $\delta = \varepsilon/8$, applies (H) with $\eta = \delta/2$, and takes $k = \lfloor x^{2\delta} \rfloor_+$.
- *Counting smooth values.* The number of possible values is bounded by $(\lg(N^k) + 1)^{\lfloor x^\delta \rfloor_+}$. The number of primes $\le y$ is bounded by $\lfloor y \rfloor_+$, and every exponent by $\lg(N^k)$.
- *Infinitude.* The source deduces unboundedness of the produced n from Lemma 1. The formal proof instead shows directly that the produced n exceeds x: n is divisible by some $p - 1$ with $p > 2x$. So the quantitative part of Lemma 1 is never needed, and is not formalized.
- *Large ε.* The case $\varepsilon \ge 1$ is reduced to $\varepsilon = 1/2$.

Lemma 3.1 (`totient_fiber_finite`)**.** *For every $n \in \mathbb{N}$ the set $\{m \in \mathbb{N} : \ 0 < m, \ \varphi(m) = n\}$ is finite.*

Lean: `totient_fiber_finite` — proved by AIProver, job 2, session 6

```
theorem totient_fiber_finite (n : ℕ) :
    {m : ℕ | 0 < m ∧ Nat.totient m = n}.Finite := by
  by_cases hn : n = 0
  · subst hn
    have h_empty : {m : ℕ | 0 < m ∧ Nat.totient m = 0} = (∅ : Set ℕ) := by
      ext m; simp; intro hm; omega
    rw [h_empty]
    exact Set.finite_empty
  have hnpos : 0 < n := Nat.pos_of_ne_zero hn

  let B : ℕ := (n + 1) ^ ((n + 2) * n)
  have hB_set : Set.Finite {m : ℕ | m ≤ B} := by
    refine Set.Finite.subset (Finset.finite_toSet (Finset.range (B + 1))) ?_
    intro m hm
    have hm_le_B : m ≤ B := hm
    have hm_lt : m < B + 1 := Nat.lt_succ_of_le hm_le_B
    exact Finset.mem_range.mpr hm_lt
  have h_bound : ∀ m, 0 < m → Nat.totient m = n → m ≤ B := by
    intro m hmpos hm_tot
    have hm_ne_zero : m ≠ 0 := by omega

    have hm_factor : m = ∏ p ∈ m.primeFactors, p ^ (m.factorization p) := by
      calc
        m = m.factorization.prod (· ^ ·) := by
```

```
      rw [Nat.factorization_prod_pow_eq_self hm_ne_zero]
    _ = ∏ p ∈ m.primeFactors, p ^ (m.factorization p) := by
      rw [Nat.prod_factorization_eq_prod_primeFactors]

  have two_pow_ge_succ : ∀ k : ℕ, k + 1 ≤ 2 ^ k := by
    intro k
    induction' k with k ih
    · simp
    · rw [pow_succ]
      have h : k + 2 ≤ 2 * (k + 1) := by omega
      calc
        k + 2 ≤ 2 * (k + 1) := h
        _ ≤ 2 * (2 ^ k) := Nat.mul_le_mul_left 2 ih
        _ = 2 ^ (k + 1) := by ring

  have h_prime_dvd : ∀ p ∈ m.primeFactors, p - 1 ∣ n := by
    intro p hp
    have hp_prime : Nat.Prime p := Nat.prime_of_mem_primeFactors hp
    let a := m.factorization p
    have ha_pos : 0 < a := by
      have h_ne_zero : m.factorization p ≠ 0 := by
        have : p ∈ (m.factorization).support := by
          rw [Nat.support_factorization]; exact hp
        rw [Finsupp.mem_support_iff] at this
        exact this
      omega
    have h_tot_pow_dvd : Nat.totient (p ^ a) ∣ Nat.totient m := by
      apply Nat.totient_dvd_of_dvd
      rw [hm_factor]
      exact Finset.dvd_prod_of_mem (λ x => x ^ (m.factorization x)) hp
    rw [hm_tot] at h_tot_pow_dvd
    have h_tot_pow_eq : Nat.totient (p ^ a) = p ^ (a - 1) * (p - 1) :=
      Nat.totient_prime_pow hp_prime ha_pos
    rw [h_tot_pow_eq] at h_tot_pow_dvd
    exact dvd_trans (dvd_mul_left _ _) h_tot_pow_dvd
  have h_prime_le : ∀ p ∈ m.primeFactors, p ≤ n + 1 := by
    intro p hp
    have hp_sub_one_dvd : p - 1 ∣ n := h_prime_dvd p hp
    have h_le : p - 1 ≤ n := Nat.le_of_dvd hnpos hp_sub_one_dvd
    omega

  have h_exponent_le : ∀ p ∈ m.primeFactors, m.factorization p ≤ n := by
    intro p hp
    have hp_prime : Nat.Prime p := Nat.prime_of_mem_primeFactors hp
    have hp_ge_two : 2 ≤ p := Nat.Prime.two_le hp_prime
    let a := m.factorization p
    have ha_pos : 0 < a := by
      have h_ne_zero : m.factorization p ≠ 0 := by
        have : p ∈ (m.factorization).support := by
          rw [Nat.support_factorization]; exact hp
        rw [Finsupp.mem_support_iff] at this
        exact this
      omega
    have h_tot_pow_dvd : Nat.totient (p ^ a) ∣ Nat.totient m := by
      apply Nat.totient_dvd_of_dvd
      rw [hm_factor]
      exact Finset.dvd_prod_of_mem (λ x => x ^ (m.factorization x)) hp
    rw [hm_tot] at h_tot_pow_dvd
    have h_tot_pow_eq : Nat.totient (p ^ a) = p ^ (a - 1) * (p - 1) :=
```

```
      Nat.totient_prime_pow hp_prime ha_pos
    rw [h_tot_pow_eq] at h_tot_pow_dvd
    have h_dvd_pow : p ^ (a - 1) ∣ n := by
      have h : p ^ (a - 1) ∣ p ^ (a - 1) * (p - 1) := dvd_mul_right _ _
      exact dvd_trans h h_tot_pow_dvd

    have ha_le_pow : a ≤ p ^ (a - 1) := by
      have h_two_ge : a ≤ 2 ^ (a - 1) := by
        have h := two_pow_ge_succ (a - 1)
        omega
      have h_pow_ge : 2 ^ (a - 1) ≤ p ^ (a - 1) := Nat.pow_le_pow_left hp_ge_two (a - 1)
      exact Nat.le_trans h_two_ge h_pow_ge
    have h_pow_le_n : p ^ (a - 1) ≤ n := Nat.le_of_dvd hnpos h_dvd_pow
    omega

  rw [hm_factor]

  have h_pow_le : ∀ p ∈ m.primeFactors, p ^ (m.factorization p) ≤ (n + 1) ^ (m.factorization p) := by
    intro p hp
    exact Nat.pow_le_pow_left (h_prime_le p hp) _
  have h_prod_le : (∏ p ∈ m.primeFactors, p ^ (m.factorization p)) ≤
                (∏ p ∈ m.primeFactors, (n + 1) ^ (m.factorization p)) :=
    Finset.prod_le_prod' h_pow_le

  have h_prod_pow : (∏ p ∈ m.primeFactors, (n + 1) ^ (m.factorization p)) =
                  (n + 1) ^ (∑ p ∈ m.primeFactors, m.factorization p) := by
    rw [Finset.prod_pow_eq_pow_sum]

  have h_card_le : m.primeFactors.card ≤ n + 2 := by
    have h_subset : m.primeFactors ⊆ Finset.range (n + 2) := by
      intro x hx
      apply Finset.mem_range.mpr
      have hx_le : x ≤ n + 1 := h_prime_le x hx
      omega
    have h_card_range : (Finset.range (n + 2)).card = n + 2 := Finset.card_range (n + 2)
    have h_card_le_range : m.primeFactors.card ≤ (Finset.range (n + 2)).card :=
      Finset.card_le_card h_subset
    calc
      m.primeFactors.card ≤ (Finset.range (n + 2)).card := h_card_le_range
      _ = n + 2 := h_card_range
  have h_sum_le : (∑ p ∈ m.primeFactors, m.factorization p) ≤ (n + 2) * n := by
    calc
      (∑ p ∈ m.primeFactors, m.factorization p) ≤ (∑ p ∈ m.primeFactors, n) :=
        Finset.sum_le_sum (λ p hp => h_exponent_le p hp)
      _ = (m.primeFactors.card : ℕ) * n := by
        rw [Finset.sum_const_nat]
        intro x hx; rfl
      _ ≤ (n + 2) * n := Nat.mul_le_mul_right n h_card_le

  have h_pow_le' : (n + 1) ^ (∑ p ∈ m.primeFactors, m.factorization p) ≤ (n + 1) ^ ((n + 2) * n) :=
    Nat.pow_le_pow_right (by omega) h_sum_le
  calc
    (∏ p ∈ m.primeFactors, p ^ (m.factorization p)) ≤
      (∏ p ∈ m.primeFactors, (n + 1) ^ (m.factorization p)) := h_prod_le
    _ = (n + 1) ^ (∑ p ∈ m.primeFactors, m.factorization p) := h_prod_pow
    _ ≤ (n + 1) ^ ((n + 2) * n) := h_pow_le'

have h_subset : {m : ℕ | 0 < m ∧ Nat.totient m = n} ⊆ {m : ℕ | m ≤ B} := by
  intro m hm
```

```
    simp [h_bound m hm.1 hm.2]
  exact hB_set.subset h_subset
```

Proof. *Case* $n = 0$. The set is empty, since $\varphi(m) = 0$ only for $m = 0$.

Case $n > 0$. Let $m > 0$ with $\varphi(m) = n$. We show $m \le B := (n+1)^{(n+2)n}$.

Write $m = \prod_{p \in \mathrm{pf}(m)} p^{a_p}$ with $a_p = v_p(m) \ge 1$. Since $p^{a_p} \mid m$, we have $\varphi(p^{a_p}) \mid \varphi(m) = n$. Also $\varphi(p^{a_p}) = p^{a_p - 1}(p-1)$. This gives two consequences:

- $p - 1 \mid n$, so $p \le n + 1$;
- $p^{a_p - 1} \mid n$, so $a_p \le 2^{a_p - 1} \le p^{a_p - 1} \le n$. Here $a \le 2^{a-1}$ for $a \ge 1$ is proved by induction.

Every prime factor of m lies in $\{0, \dots, n+1\}$, so $|\mathrm{pf}(m)| \le n + 2$. Hence

$$m = \prod_p p^{a_p} \le \prod_p (n+1)^{a_p} = (n+1)^{\sum_p a_p} \le (n+1)^{(n+2)n} = B.$$

The fiber is therefore contained in the finite set $\{0, \dots, B\}$. □

Lemma 3.2 (`card_le_totientFiberCard`)**.** *Let* $n \in \mathbb{N}$ *and let* $T \subset \mathbb{N}$ *be finite. Suppose every* $m \in T$ *satisfies* $0 < m$ *and* $\varphi(m) = n$*. Then* $|T| \le g(n)$*.*

Lean: `card_le_totientFiberCard` — proved by AIProver, job 2, session 1

```
theorem card_le_totientFiberCard (n : ℕ) (T : Finset ℕ)
    (hT : ∀ m ∈ T, 0 < m ∧ Nat.totient m = n) :
    T.card ≤ totientFiberCard n := by
  have h_fin : {m : ℕ | 0 < m ∧ Nat.totient m = n}.Finite := totient_fiber_finite n
  haveI : Fintype {m : ℕ // 0 < m ∧ Nat.totient m = n} := h_fin.fintype
  have h_card_eq : totientFiberCard n = Fintype.card {m : ℕ // 0 < m ∧ Nat.totient m = n} := by
    simp [totientFiberCard, Nat.card_eq_fintype_card]
  rw [h_card_eq]
  let f : T → {m : ℕ // 0 < m ∧ Nat.totient m = n} := λ m => ⟨m.1, hT m.1 m.2⟩
  have hf_inj : Function.Injective f := by
    intro a b h
    have hval : (f a).1 = (f b).1 := by rw [h]
    apply Subtype.ext
    simpa using hval
  let S : Finset T := Finset.univ
  have h_card_T : S.card = T.card := by
    calc
      S.card = Fintype.card {x // x ∈ T} := by simp [S]
      _ = T.card := by simp
  have h_image_card : (Finset.image f S).card = S.card :=
    Finset.card_image_of_injective S hf_inj
  have h_sub : Finset.image f S ⊆ (Finset.univ : Finset {m : ℕ // 0 < m ∧ Nat.totient m = n}) :=
    Finset.subset_univ _
  have h_card_le : (Finset.image f S).card ≤ (Finset.univ : Finset {m : ℕ // 0 < m ∧ Nat.totient m =
↪  n}).card :=
    Finset.card_le_card h_sub
  have h_univ_card : (Finset.univ : Finset {m : ℕ // 0 < m ∧ Nat.totient m = n}).card = Fintype.card {m : ℕ
↪  // 0 < m ∧ Nat.totient m = n} := by
    simp
  calc
    T.card = S.card := by symm; exact h_card_T
    _ = (Finset.image f S).card := by symm; exact h_image_card
    _ ≤ (Finset.univ : Finset {m : ℕ // 0 < m ∧ Nat.totient m = n}).card := h_card_le
    _ = Fintype.card {m : ℕ // 0 < m ∧ Nat.totient m = n} := h_univ_card
```

Proof. By Lemma 3.1 the fiber is finite, so $g(n)$ is its cardinality. The inclusion of T into the fiber is injective, which gives $|T| \le g(n)$. □

Lemma 3.3 (totient_prod_primes). *Let $Q \subset \mathbb{N}$ be a finite set of primes. Then $\varphi(\prod_{p\in Q} p) = \prod_{p\in Q}(p-1)$.*

Lean: totient_prod_primes — proved by AIProver, job 1, session 1

```
theorem totient_prod_primes (Q : Finset ℕ) (hQ : ∀ p ∈ Q, p.Prime) :
    Nat.totient (∏ p ∈ Q, p) = ∏ p ∈ Q, (p - 1) := by
  induction' Q using Finset.induction_on with p Q hp ih
  · simp
  · have hp_prime : p.Prime := hQ p (Finset.mem_insert_self p Q)
    have hQ' : ∀ q ∈ Q, q.Prime := fun q hq => hQ q (Finset.mem_insert_of_mem hq)
    rw [Finset.prod_insert hp, Finset.prod_insert hp]
    have hcop : (∏ x ∈ Q, x).Coprime p := by
      refine Nat.Coprime.prod_left fun q hq => ?_
      have hq_prime : q.Prime := hQ' q hq
      refine hq_prime.coprime_iff_not_dvd.mpr fun h => ?_
      have hq_one_lt : 1 < q := hq_prime.one_lt
      have hp_eq_q : p = q := ((hp_prime.dvd_iff_eq hq_one_lt.ne').mp h)
      exact hp (hp_eq_q ▸ hq)
    rw [← mul_comm, Nat.totient_mul hcop, mul_comm, ih hQ', Nat.totient_prime hp_prime, mul_comm]
```

Proof. We use induction on Q. The empty case is $\varphi(1) = 1$.

For $Q \cup \{p\}$ with $p \notin Q$: the product $\prod_{q\in Q} q$ is coprime to p, because each $q \in Q$ is a prime different from p. By multiplicativity of φ, the induction hypothesis, and $\varphi(p) = p - 1$,

$$\varphi\Big(p \prod_{q\in Q} q\Big) = (p-1) \prod_{q\in Q} (q-1).$$

□

Lemma 3.4 (prod_primes_inj). *Let $Q_1, Q_2 \subset \mathbb{N}$ be finite sets of primes with $\prod_{p\in Q_1} p = \prod_{p\in Q_2} p$. Then $Q_1 = Q_2$.*

Lean: prod_primes_inj — proved by AIProver, job 1, session 0

```
theorem prod_primes_inj (Q₁ Q₂ : Finset ℕ) (h₁ : ∀ p ∈ Q₁, p.Prime)
    (h₂ : ∀ p ∈ Q₂, p.Prime) (h : ∏ p ∈ Q₁, p = ∏ p ∈ Q₂, p) : Q₁ = Q₂ := by
  calc
    Q₁ = Nat.primeFactors (∏ p ∈ Q₁, p) := by
      rw [Nat.primeFactors_prod h₁]
    _ = Nat.primeFactors (∏ p ∈ Q₂, p) := by rw [h]
    _ = Q₂ := by rw [Nat.primeFactors_prod h₂]
```

Proof. For a finite set Q of primes, $\mathrm{pf}(\prod_{p\in Q} p) = Q$ (Mathlib's Nat.primeFactors_prod). Hence

$$Q_1 = \mathrm{pf}\Big(\prod_{Q_1} p\Big) = \mathrm{pf}\Big(\prod_{Q_2} p\Big) = Q_2.$$

□

Lemma 3.5 (exists_large_fiber). *Let $S \subset \mathbb{N}$ be a finite set of primes, let $k \in \mathbb{N}$ with $k \le |S|$, and let $V \subset \mathbb{N}$ be finite. Suppose that for every $Q \subseteq S$ with $|Q| = k$ we have $\prod_{p\in Q}(p-1) \in V$. Then there is $Q \subseteq S$ with $|Q| = k$ and*

$$\binom{|S|}{k} \le |V| \cdot g\Big(\prod_{p\in Q} (p-1)\Big).$$

Lean: `exists_large_fiber` — proved by the captain after a hand-back

```
theorem exists_large_fiber (S : Finset ℕ) (hS : ∀ p ∈ S, p.Prime) (k : ℕ)
    (hk : k ≤ S.card) (V : Finset ℕ)
    (hV : ∀ Q ⊆ S, Q.card = k → ∏ p ∈ Q, (p - 1) ∈ V) :
    ∃ Q ⊆ S, Q.card = k ∧
      S.card.choose k ≤ V.card * totientFiberCard (∏ p ∈ Q, (p - 1)) := by
  let T := Finset.powersetCard k S
  have hTcard : T.card = S.card.choose k := by simp [T]
  let f : Finset ℕ → ℕ := fun Q => ∏ p ∈ Q, (p - 1)
  have hf_maps_to : ∀ Q ∈ T, f Q ∈ V := by
    intro Q hQ
    rcases Finset.mem_powersetCard.mp hQ with ⟨hQsub, hQcard⟩
    exact hV Q hQsub hQcard
  have h_sum_eq : T.card = ∑ n ∈ V, (Finset.filter (fun Q => f Q = n) T).card := by
    have h_mapsTo : Set.MapsTo f (T : Set (Finset ℕ)) (V : Set ℕ) := by
      intro Q hQ
      simp [hf_maps_to Q hQ]
    simpa using Finset.card_eq_sum_card_fiberwise h_mapsTo
  by_cases hVempty : V = ∅
  · rw [hVempty] at hf_maps_to
    have hTempty : T = ∅ := by
      by_contra hTnonempty
      obtain ⟨Q, hQ⟩ := Finset.nonempty_iff_ne_empty.mpr hTnonempty
      have h_in_V := hf_maps_to Q hQ
      simp at h_in_V
    rw [hTempty] at hTcard
    simp at hTcard
    have hpos : 0 < S.card.choose k := Nat.choose_pos hk
    rw [hTcard] at hpos
    exact absurd hpos (lt_irrefl _)
  · have hVnonempty : V.Nonempty :=
      Finset.nonempty_iff_ne_empty.mpr hVempty
    have h_exists_max : ∃ n ∈ V, ∀ n' ∈ V,
      (Finset.filter (fun Q => f Q = n') T).card ≤ (Finset.filter (fun Q => f Q = n) T).card := by
      simpa using Finset.exists_max_image V (fun n => (Finset.filter (fun Q => f Q = n) T).card) hVnonempty
    rcases h_exists_max with ⟨n, hnV, hn_max⟩
    have h_sum_le : ∑ n' ∈ V, (Finset.filter (fun Q => f Q = n') T).card ≤
      V.card * (Finset.filter (fun Q => f Q = n) T).card := by
      calc
        ∑ n' ∈ V, (Finset.filter (fun Q => f Q = n') T).card
            ≤ ∑ n' ∈ V, (Finset.filter (fun Q => f Q = n) T).card :=
          Finset.sum_le_sum (fun n' hn' => hn_max n' hn')
        _ = V.card * (Finset.filter (fun Q => f Q = n) T).card := by simp
    have h_ineq : T.card ≤ V.card * (Finset.filter (fun Q => f Q = n) T).card := by
      linarith
    let fiber := Finset.filter (fun Q => f Q = n) T
    by_cases h_fiber_card_zero : fiber.card = 0
    · rw [h_fiber_card_zero, mul_zero] at h_ineq
      have hTpos : 0 < T.card := by
        rw [hTcard]
        exact Nat.choose_pos hk
      have : ¬ T.card ≤ 0 := Nat.not_le_of_lt hTpos
      exact absurd h_ineq this
    · have h_fiber_card_pos : 0 < fiber.card :=
        Nat.pos_of_ne_zero h_fiber_card_zero
      have h_fiber_nonempty : fiber.Nonempty := Finset.card_pos.mp h_fiber_card_pos
      obtain ⟨Q, hQ⟩ := h_fiber_nonempty
      refine ⟨Q, ?_, ?_, ?_⟩
```

```
    · rcases Finset.mem_filter.mp hQ with ⟨hQT, hQf⟩
      rcases Finset.mem_powersetCard.mp hQT with ⟨hQsub, hQcard⟩
      exact hQsub
    · rcases Finset.mem_filter.mp hQ with ⟨hQT, hQf⟩
      rcases Finset.mem_powersetCard.mp hQT with ⟨hQsub, hQcard⟩
      exact hQcard
    · have hfQ : f Q = n := by
        rcases Finset.mem_filter.mp hQ with ⟨_, hQf⟩
        exact hQf
      rw [← hTcard, show totientFiberCard (∏ p ∈ Q, (p - 1)) = totientFiberCard (f Q) by
        simp [f], hfQ]
      apply le_trans h_ineq
      apply Nat.mul_le_mul_left V.card
      have h_card_le : fiber.card ≤ totientFiberCard n := by
        have hmem : ∀ Q ∈ fiber, Q ⊆ S ∧ Q.card = k ∧ ∏ p ∈ Q, (p - 1) = n := by
          intro Q hQ
          rcases Finset.mem_filter.mp hQ with ⟨hQT, hQf⟩
          rcases Finset.mem_powersetCard.mp hQT with ⟨hQsub, hQcard⟩
          exact ⟨hQsub, hQcard, hQf⟩
        have hinj : Set.InjOn (fun Q : Finset ℕ => ∏ p ∈ Q, p) (fiber : Set (Finset ℕ)) := by
          intro Q1 hQ1 Q2 hQ2 h
          have h1 := hmem Q1 (Finset.mem_coe.mp hQ1)
          have h2 := hmem Q2 (Finset.mem_coe.mp hQ2)
          exact prod_primes_inj Q1 Q2 (fun p hp => hS p (h1.1 hp))
            (fun p hp => hS p (h2.1 hp)) h
        calc fiber.card = (fiber.image (fun Q : Finset ℕ => ∏ p ∈ Q, p)).card :=
              (Finset.card_image_of_injOn hinj).symm
          _ ≤ totientFiberCard n := by
            apply card_le_totientFiberCard
            intro m hm
            obtain ⟨Q', hQ', rfl⟩ := Finset.mem_image.mp hm
            have hQ'' := hmem Q' hQ'
            show 0 < ∏ p ∈ Q', p ∧ Nat.totient (∏ p ∈ Q', p) = n
            refine ⟨Finset.prod_pos (fun p hp => (hS p (hQ''.1 hp)).pos), ?_⟩
            rw [totient_prod_primes Q' (fun p hp => hS p (hQ''.1 hp))]
            exact hQ''.2.2
      exact h_card_le
```

Proof. Let T be the family of k-subsets of S, so $|T| = \binom{|S|}{k} \geq 1$ since $k \leq |S|$. Let $f(Q) = \prod_{p \in Q}(p-1)$. By hypothesis f maps T into V. Hence

$$|T| = \sum_{v \in V} |T_v|, \qquad T_v = \{Q \in T : f(Q) = v\}.$$

The set V is nonempty: otherwise T would be empty, contradicting $|T| \geq 1$. Choose $n \in V$ with $|T_n|$ maximal. Then $|T| \leq |V|\,|T_n|$. In particular $T_n \neq \emptyset$, since $|T| > 0$. Pick $Q \in T_n$, so $f(Q) = n$.

It remains to show $|T_n| \leq g(n)$. The map $Q' \mapsto \prod_{p \in Q'} p$ is injective on T_n by Lemma 3.4. Each image is positive, and by Lemma 3.3 its totient is $f(Q') = n$. By Lemma 3.2 the image has at most $g(n)$ elements, so $|T_n| \leq g(n)$. Therefore

$$\binom{|S|}{k} \leq |V|\, g(n). \qquad \square$$

Lemma 3.6 (`smooth_count`). *Let $N \in \mathbb{N}$ and $y \in \mathbb{R}$. There is a finite set $V \subset \mathbb{N}$ with*

$$|V| \leq (\lg(N) + 1)^{\lfloor y \rfloor_+}$$

such that every $n \in \mathbb{N}$ with $0 < n \le N$ that is y-smooth lies in V.

Lean: `smooth_count` — proved by the captain after a hand-back

```
theorem smooth_count (N : ℕ) (y : ℝ) :
    ∃ V : Finset ℕ, V.card ≤ (Nat.log 2 N + 1) ^ y ∧
      ∀ n : ℕ, 0 < n → n ≤ N → IsSmoothNat n y → n ∈ V := by
  classical
  obtain ⟨P, hP⟩ : ∃ P : Finset ℕ, P = (Finset.Icc 1 y).filter Nat.Prime := ⟨_, rfl⟩
  have hPprime : ∀ q ∈ P, q.Prime := by
    intro q hq
    rw [hP, Finset.mem_filter] at hq
    exact hq.2
  refine ⟨(Fintype.piFinset (fun _ : P => Finset.range (Nat.log 2 N + 1))).image
      (fun f => ∏ p : P, (p : ℕ) ^ (f p)), ?_, ?_⟩
  · calc _ ≤ (Fintype.piFinset (fun _ : P => Finset.range (Nat.log 2 N + 1))).card :=
          Finset.card_image_le
      _ = (Nat.log 2 N + 1) ^ P.card := by
          rw [Fintype.card_piFinset, Finset.prod_const, Finset.card_univ, Fintype.card_coe,
            Finset.card_range]
      _ ≤ (Nat.log 2 N + 1) ^ y := by
          apply Nat.pow_le_pow_right (by omega)
          calc P.card ≤ (Finset.Icc 1 y).card := by
                rw [hP]; exact Finset.card_filter_le _ _
            _ = y := by simp
  · intro n hn hnN hsm
    rw [Finset.mem_image]
    refine ⟨fun p => n.factorization p, ?_, ?_⟩
    · rw [Fintype.mem_piFinset]
      intro p
      rw [Finset.mem_range, Nat.lt_add_one_iff]
      apply Nat.le_log_of_pow_le (by norm_num)
      have hp : (p : ℕ).Prime := hPprime p.1 p.2
      calc 2 ^ n.factorization p ≤ (p : ℕ) ^ n.factorization p :=
            Nat.pow_le_pow_left hp.two_le _
        _ ≤ n := Nat.le_of_dvd hn
            ((Nat.Prime.pow_dvd_iff_le_factorization hp hn.ne').mpr le_rfl)
        _ ≤ N := hnN
    · show ∏ p : P, (p : ℕ) ^ n.factorization p = n
      have hc := Finset.prod_coe_sort P (fun p => p ^ n.factorization p)
      simp only at hc
      rw [hc]
      conv_rhs => rw [← Nat.factorization_prod_pow_eq_self hn.ne']
      rw [Finsupp.prod, Nat.support_factorization]
      symm
      apply Finset.prod_subset
      · intro q hq
        rw [hP, Finset.mem_filter, Finset.mem_Icc]
        have hq' := Nat.prime_of_mem_primeFactors hq
        exact ⟨⟨hq'.one_lt.le, Nat.le_floor (hsm q hq)⟩, hq'⟩
      · intro q _ hq
        show q ^ n.factorization q = 1
        have h0 : n.factorization q = 0 := by
          by_contra h
          exact hq (by rw [← Nat.support_factorization, Finsupp.mem_support_iff]; exact h)
        rw [h0, pow_zero]
```

Proof. Let P be the set of primes in $[1, \lfloor y \rfloor_+]$, so $|P| \le \lfloor y \rfloor_+$. Let V be the image of the map

$$e \longmapsto \prod_{p \in P} p^{e(p)}, \qquad e : P \to \{0, \dots, \lg(N)\}.$$

Then $|V| \le (\lg(N)+1)^{|P|} \le (\lg(N)+1)^{\lfloor y \rfloor_+}$.

Now let $0 < n \le N$ be y-smooth.

- For $p \in P$ we have $2^{v_p(n)} \le p^{v_p(n)} \le n \le N$, hence $v_p(n) \le \lg(N)$.
- Every prime factor q of n satisfies $q \le y$. Since q is an integer, $q \le \lfloor y \rfloor_+$, so $q \in P$.
- Primes $p \in P$ that do not divide n have $v_p(n) = 0$.

Hence $n = \prod_{p \in \mathrm{pf}(n)} p^{v_p(n)} = \prod_{p \in P} p^{v_p(n)} \in V$. □

Lemma 3.7 (prod_smooth_bounded)**.** *Let $Q \subset \mathbb{N}$ be finite, $y \in \mathbb{R}$, $B \in \mathbb{N}$. Suppose each $p \in Q$ satisfies*

- $0 < p - 1$ *(truncated subtraction),*
- $p - 1 \le B$*,*
- $p - 1$ *is y-smooth.*

Then $0 < \prod_{p \in Q}(p-1) \le B^{|Q|}$, and $\prod_{p \in Q}(p-1)$ is y-smooth.

Lean: prod_smooth_bounded — proved by AIProver, job 1, session 6

```
theorem prod_smooth_bounded (Q : Finset ℕ) (y : ℝ) (B : ℕ)
    (h : ∀ p ∈ Q, 0 < p - 1 ∧ p - 1 ≤ B ∧ IsSmoothNat (p - 1) y) :
    0 < ∏ p ∈ Q, (p - 1) ∧ ∏ p ∈ Q, (p - 1) ≤ B ^ Q.card ∧
      IsSmoothNat (∏ p ∈ Q, (p - 1)) y := by
  have hpos : ∀ p ∈ Q, 0 < p - 1 := λ p hp => (h p hp).1
  have hle : ∀ p ∈ Q, p - 1 ≤ B := λ p hp => (h p hp).2.1
  have hsm : ∀ p ∈ Q, IsSmoothNat (p - 1) y := λ p hp => (h p hp).2.2
  have hpos_prod : 0 < ∏ p ∈ Q, (p - 1) := by
    refine Finset.prod_pos ?_
    intro p hp
    exact hpos p hp
  have h_le_prod : ∏ p ∈ Q, (p - 1) ≤ B ^ Q.card := by
    calc
      ∏ p ∈ Q, (p - 1) ≤ ∏ p ∈ Q, B := Finset.prod_le_prod' (by
        intro p hp
        have hp_pos := hpos p hp
        have hble := hle p hp
        have : (p - 1 : ℕ) ≤ B := hble
        exact this
      )
      _ = B ^ Q.card := by simp
  have hnz : ∏ p ∈ Q, (p - 1) ≠ 0 := by
    have : 0 < ∏ p ∈ Q, (p - 1) := hpos_prod
    omega
  have hsm_prod : IsSmoothNat (∏ p ∈ Q, (p - 1)) y := by
    intro q hq
    have hq_mem_ps : q ∈ (∏ p ∈ Q, (p - 1)).primeFactors := hq
    have hq_prime : q.Prime := Nat.prime_of_mem_primeFactors hq_mem_ps
```

```
    have hq_dvd : q ∣ ∏ p ∈ Q, (p - 1) := Nat.dvd_of_mem_primeFactors hq_mem_ps

    rcases (Prime.dvd_finset_prod_iff (hq_prime.prime) (λ p => p - 1)).mp hq_dvd with ⟨p, hp, hq_dvd_p⟩
    have hsm_p := hsm p hp

    have hq_mem_factor : q ∈ (p - 1).primeFactors := by
      have hp_pos := hpos p hp
      have hp1_ne_zero : p - 1 ≠ 0 := by omega
      rw [Nat.mem_primeFactors_of_ne_zero hp1_ne_zero]
      exact ⟨hq_prime, hq_dvd_p⟩

    exact hsm_p q hq_mem_factor
  exact And.intro hpos_prod (And.intro h_le_prod hsm_prod)
```

Proof. Positivity and the bound $\prod(p-1) \le \prod B = B^{|Q|}$ are immediate.

For smoothness, let q be a prime factor of the product. Since q is prime, it divides some factor $p-1$ with $p \in Q$. As $p-1 \neq 0$, q is a prime factor of $p-1$, so $q \le y$. □

Lemma 3.8 (choose_ge_pow_div_aiprover_s9). *For $M, k \in \mathbb{N}$ with $k \le M$, $(M/k)^k \le \binom{M}{k}$ in $\mathbb{R}$.*

Lean: choose_ge_pow_div_aiprover_s9 — helper of choose_ge_pow_div, proved by AIProver, job 1, session 9

```
open Finset Nat in
theorem choose_ge_pow_div_aiprover_s9 (M k : ℕ) (hk : k ≤ M) :
    ((M : ℝ) / k) ^ k ≤ (M.choose k : ℝ) := by
  by_cases hkz : k = 0
  · subst hkz; simp
  · have hkpos_nat : 0 < k := Nat.pos_of_ne_zero hkz
    have hkpos' : (k : ℝ) > 0 := by exact mod_cast hkpos_nat



    have h_factor (j : ℕ) (hj : j < k) : (M : ℝ) - (j : ℝ) ≥ ((M : ℝ) / k) * ((k : ℝ) - (j : ℝ)) := by
      have hkM : (k : ℝ) ≤ (M : ℝ) := by exact mod_cast hk
      have hMk : (M : ℝ) / k ≥ 1 := ((one_le_div hkpos').mpr hkM)
      calc
        (M : ℝ) - (j : ℝ) = (M : ℝ) - (j : ℝ) := rfl
        _ ≥ (M : ℝ) - (M : ℝ) / k * (j : ℝ) := by
          have : (M : ℝ) / k * (j : ℝ) ≥ 1 * (j : ℝ) := by nlinarith
          nlinarith
        _ = (M : ℝ) / k * k - (M : ℝ) / k * (j : ℝ) := by
          field_simp [hkpos'.ne']
        _ = ((M : ℝ) / k) * ((k : ℝ) - (j : ℝ)) := by ring


    have h_desc_fact_ℕ : M.descFactorial k = k ! * M.choose k := by
      rw [descFactorial_eq_factorial_mul_choose]

    have h_prod_desc_ℕ : M.descFactorial k = ∏ i ∈ range k, (M - i) := by
      rw [descFactorial_eq_prod_range]


    have h_prod_desc_ℝ : (M.descFactorial k : ℝ) = ∏ i ∈ range k, ((M : ℝ) - (i : ℝ)) := by
      have h := congrArg (Nat.cast : ℕ → ℝ) h_prod_desc_ℕ

      rw [h, Nat.cast_prod]
```

```
    refine Finset.prod_congr rfl (fun i hi => ?_)
    have hi_lt_k : i < k := Finset.mem_range.1 hi
    have hiM : i ≤ M := Nat.le_trans (Nat.le_of_lt hi_lt_k) hk
    rw [Nat.cast_sub hiM]

  have h_desc_fact_ℝ : (M.descFactorial k : ℝ) = (k ! : ℝ) * (M.choose k : ℝ) := by
    exact mod_cast h_desc_fact_ℕ


  have h_choose_eq : (M.choose k : ℝ) = (∏ i ∈ range k, ((M : ℝ) - (i : ℝ))) / (k ! : ℝ) := by
    have hk_fact_ne_zero : (k ! : ℝ) ≠ 0 := by exact mod_cast factorial_ne_zero k
    field_simp [hk_fact_ne_zero]
    nlinarith


  have h_prod_factor_ge : (∏ i ∈ range k, ((M : ℝ) - (i : ℝ))) ≥
      (∏ i ∈ range k, (((M : ℝ) / k) * ((k : ℝ) - (i : ℝ)))) := by
    apply Finset.prod_le_prod
    · intro i hi
      have hi_lt_k : i < k := Finset.mem_range.1 hi
      have h_nonneg_k_sub_i : 0 ≤ (k : ℝ) - (i : ℝ) := by
        have : (i : ℝ) ≤ (k : ℝ) := by exact mod_cast hi_lt_k.le
        nlinarith
      have hM_nonneg : 0 ≤ (M : ℝ) := by exact mod_cast Nat.zero_le M
      have h_div_nonneg : 0 ≤ (M : ℝ) / k := div_nonneg hM_nonneg (by positivity)
      exact mul_nonneg h_div_nonneg h_nonneg_k_sub_i
    · intro i hi
      have hi_lt_k : i < k := Finset.mem_range.1 hi
      exact h_factor i hi_lt_k


  have h_prod_split : (∏ i ∈ range k, (((M : ℝ) / k) * ((k : ℝ) - (i : ℝ)))) =
      ((M : ℝ) / k) ^ k * (k ! : ℝ) := by
    calc
      (∏ i ∈ range k, (((M : ℝ) / k) * ((k : ℝ) - (i : ℝ)))) =
          (∏ i ∈ range k, ((M : ℝ) / k)) * (∏ i ∈ range k, ((k : ℝ) - (i : ℝ))) := by
        rw [Finset.prod_mul_distrib]
      _ = (((M : ℝ) / k) ^ k) * (∏ i ∈ range k, ((k : ℝ) - (i : ℝ))) := by

        simp [div_pow]
      _ = (((M : ℝ) / k) ^ k) * ((k.descFactorial k : ℝ)) := by
        rw [descFactorial_eq_prod_range, Nat.cast_prod]
        refine congrArg (fun p : ℝ => ((M : ℝ) / k) ^ k * p) (Finset.prod_congr rfl (fun i hi => ?_))
        have hi_lt_k : i < k := Finset.mem_range.1 hi
        rw [Nat.cast_sub (Nat.le_of_lt hi_lt_k)]
      _ = ((M : ℝ) / k) ^ k * (k ! : ℝ) := by rw [descFactorial_self]


  have h_main : ((M : ℝ) / k) ^ k * (k ! : ℝ) ≤ (∏ i ∈ range k, ((M : ℝ) - (i : ℝ))) := by
    nlinarith


  rw [h_choose_eq]
  have hk_fact_pos : (0 : ℝ) < k ! := by exact mod_cast factorial_pos k
  exact (le_div_iff₀ hk_fact_pos).mpr h_main
```

Proof. For $k = 0$ both sides equal 1, using the convention $M/0 = 0$ and $0^0 = 1$.

Let $k \geq 1$. Then $M/k \geq 1$, so for $0 \leq j < k$,

$$M - j \;\geq\; M - \frac{M}{k}\, j = \frac{M}{k}\,(k - j).$$

Both sides are nonnegative. Multiplying over $j < k$ gives

$$k!\binom{M}{k} = \prod_{j<k}(M - j) \;\geq\; \Big(\frac{M}{k}\Big)^{k} \prod_{j<k}(k - j) = \Big(\frac{M}{k}\Big)^{k} k!.$$

Dividing by $k!$ gives the claim. This is the source's argument, with each factor of $\prod_j (M-j)/(k-j)$ at least M/k. □

Lemma 3.9 (choose_ge_pow_div). *For $M, k \in \mathbb{N}$ with $k \leq M$, $(M/k)^k \leq \binom{M}{k}$ in $\mathbb{R}$.*

Lean: choose_ge_pow_div — proved by AIProver, job 1, session 9

```
theorem choose_ge_pow_div (M k : ℕ) (hk : k ≤ M) :
    ((M : ℝ) / k) ^ k ≤ (M.choose k : ℝ) := by
  first
    | exact choose_ge_pow_div_aiprover_s9
    | (intros; apply choose_ge_pow_div_aiprover_s9 <;> assumption)
```

Proof. This is Lemma 3.8. □

Lemma 3.10 (asymp_log_ineq_growth_logbound). *For real $\delta > 0$ and $x \geq 1$,*

$$\log\Big(\frac{x^{2\delta}\log(5x)}{\log 2} + 1\Big) \leq \log 11 + (2\delta + 1)\log x.$$

Lean: asymp_log_ineq_growth_logbound — proved by AIProver, job 1, session 0

```
theorem asymp_log_ineq_growth_logbound (δ x : ℝ) (hδ : 0 < δ) (hx : 1 ≤ x) :
    Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1)
      ≤ Real.log 11 + (2 * δ + 1) * Real.log x := by
  have hxpos : 0 < x := by linarith
  have hlog2_pos : 0 < Real.log 2 := Real.log_pos (by norm_num : (1 : ℝ) < 2)
  have hxp_pos : 0 < x ^ (2 * δ) := Real.rpow_pos_of_pos hxpos _
  have hlogx_nonneg : 0 ≤ Real.log x := Real.log_nonneg hx

  have h_log5_lt_2 : Real.log 5 < 2 := by
    have h_exp2_gt_5 : (5 : ℝ) < Real.exp 2 := by
      have h_sum_le_exp2 : (∑ m ∈ Finset.range 8, (2 : ℝ) ^ m / m.factorial) ≤ Real.exp 2 :=
        Real.sum_le_exp_of_nonneg (by norm_num) 8
      have h_sum_gt_5 : (5 : ℝ) < (∑ m ∈ Finset.range 8, (2 : ℝ) ^ m / m.factorial) := by
        have h_eq : (∑ m ∈ Finset.range 8, (2 : ℝ) ^ m / m.factorial) =
            155/21 := by
          simp [Finset.sum_range_succ]
          ring
        rw [h_eq]
        norm_num
      linarith
    have h_log_eq : Real.log (Real.exp 2) = 2 := Real.log_exp _
    have h_log_lt : Real.log 5 < Real.log (Real.exp 2) :=
      Real.log_lt_log (by norm_num : 0 < (5 : ℝ)) h_exp2_gt_5
```

```
    linarith

  have h_10log2_gt_3 : 3 < 10 * Real.log 2 := by
    have h_log2_gt_3_10 : 3/10 < Real.log 2 := by
      have h_exp_lt_2' : Real.exp (3/10) < 2 := by
        have h_sum_lt_2 : (∑ m ∈ Finset.range 8, ((3/10 : ℝ)) ^ m / m.factorial) < 2 := by
          have h_eq : (∑ m ∈ Finset.range 8, ((3/10 : ℝ)) ^ m / m.factorial) =
              1 + 3/10 + ((3/10)^2)/2 + ((3/10)^3)/6 + ((3/10)^4)/24 +
              ((3/10)^5)/120 + ((3/10)^6)/720 + ((3/10)^7)/5040 := by
            simp [Finset.sum_range_succ]
            ring
          rw [h_eq]
          norm_num
        have h_upper : Real.exp (3/10) ≤ (∑ m ∈ Finset.range 8, ((3/10 : ℝ)) ^ m / m.factorial) +
          ((3/10 : ℝ)) ^ 8 * (8 + 1) / ((8 : ℕ).factorial * 8) := by
          refine Real.exp_bound' (by norm_num) (by linarith) (by norm_num)
        have h_rem_lt : ((3/10 : ℝ)) ^ 8 * (8 + 1) / ((8 : ℕ).factorial * 8) < 1 := by
          have hpos : 0 < ((3/10 : ℝ)) ^ 8 * (8 + 1) / ((8 : ℕ).factorial * 8) := by positivity
          have hlt : ((3/10 : ℝ)) ^ 8 * (8 + 1) / ((8 : ℕ).factorial * 8) < 0.0001 := by
            nlinarith
          nlinarith
        nlinarith
      have h_log_eq' : Real.log (Real.exp (3/10)) = 3/10 := Real.log_exp _
      have h_log_lt' : Real.log (Real.exp (3/10)) < Real.log 2 :=
        Real.log_lt_log (Real.exp_pos _) h_exp_lt_2'
      linarith
    nlinarith

  have h_log5x_le_10x : Real.log (5 * x) ≤ 10 * Real.log 2 * x := by
    rw [Real.log_mul (by norm_num : (5 : ℝ) ≠ 0) hxpos.ne']
    have h_logx_le_x : Real.log x ≤ x := by
      have := Real.log_le_sub_one_of_pos hxpos
      linarith
    calc
      Real.log 5 + Real.log x ≤ 2 + Real.log x := by linarith
      _ ≤ 2 + x := by nlinarith
      _ ≤ 10 * Real.log 2 * x := by
        have h : 2 ≤ (10 * Real.log 2 - 1) * 1 := by
          nlinarith
        nlinarith

  have h_main : Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1)
      ≤ Real.log 11 + (2 * δ + 1) * Real.log x := by
    have h_split : Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1) =
        Real.log (x ^ (2 * δ)) + Real.log (Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹) := by
      have hpos_inner : 0 < Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹ := by
        have hlog5x_pos : 0 < Real.log (5 * x) := Real.log_pos (by
          have : 1 < 5 * x := by nlinarith
          exact this)
        positivity
      have h_eq : x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1 =
          x ^ (2 * δ) * (Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹) := by
        field_simp [hlog2_pos.ne']
      rw [h_eq]
      exact calc
        Real.log (x ^ (2 * δ) * (Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹))
            = Real.log (x ^ (2 * δ)) + Real.log (Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹) :=
          Real.log_mul (hxp_pos.ne') (hpos_inner.ne')
        _ = Real.log (x ^ (2 * δ)) + Real.log (Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹) := rfl
```

```
    rw [h_split]
    have h_log_xpow : Real.log (x ^ (2 * δ)) = (2 * δ) * Real.log x := by
      rw [Real.log_rpow hxpos]
    rw [h_log_xpow]




    have h_inner_bound : Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹ ≤ 11 * x := by
      have h_main_bound : Real.log (5 * x) / Real.log 2 ≤ 10 * x := by
        rw [div_le_iff₀ hlog2_pos]

        simpa [mul_comm, mul_left_comm, mul_assoc] using h_log5x_le_10x
      have h_inv_le_one : (x ^ (2 * δ))⁻¹ ≤ 1 := by
        have hxp_ge_one : 1 ≤ x ^ (2 * δ) := by
          have : 0 ≤ 2 * δ := by linarith
          calc
            1 = x ^ (0 : ℝ) := by simp
            _ ≤ x ^ (2 * δ) := Real.rpow_le_rpow_of_exponent_le hx this
        have h := (inv_le_inv₀ (by positivity) (by positivity)).2 hxp_ge_one
        simpa using h
      calc
        Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹ ≤ 10 * x + (x ^ (2 * δ))⁻¹ := by
          nlinarith
        _ ≤ 10 * x + 1 := by nlinarith
        _ ≤ 11 * x := by nlinarith
    have h_inner_pos : 0 < Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹ := by
      have hlog5x_pos : 0 < Real.log (5 * x) := Real.log_pos (by
        have : 1 < 5 * x := by nlinarith
        exact this)
      positivity
    have h_log_inner : Real.log (Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹)
        ≤ Real.log (11 * x) := by
      exact Real.log_le_log h_inner_pos h_inner_bound
    have h_log_11x : Real.log (11 * x) = Real.log 11 + Real.log x := by
      rw [Real.log_mul (by norm_num : (11 : ℝ) ≠ 0) hxpos.ne']
    rw [h_log_11x] at h_log_inner
    nlinarith
  exact h_main
```

Proof. We first establish two numerical facts.

- $\log 5 < 2$. Indeed $e^2 \geq \sum_{m<8} 2^m/m! = 155/21 > 5$.
- $\log 2 > 3/10$. Indeed $e^{3/10}$ is at most $\sum_{m<8}(3/10)^m/m!$ plus the Taylor remainder bound $(3/10)^8 \cdot 9/(8! \cdot 8)$, and this total is < 2.

From $\log x \leq x - 1$ and $10 \log 2 - 1 \geq 2$ we get

$$\log(5x) = \log 5 + \log x \leq 2 + x \leq 10 \log 2 \cdot x.$$

Therefore $\log(5x)/\log 2 \leq 10x$. Since $x^{2\delta} \geq 1$ we also have $x^{-2\delta} \leq 1$.

Now write the argument of the logarithm as

$$x^{2\delta}\Big(\frac{\log(5x)}{\log 2} + x^{-2\delta}\Big).$$

The second factor lies in $(0, 11x]$. Taking logarithms,

$$\log(\cdots) \le 2\delta \log x + \log(11x) = \log 11 + (2\delta + 1)\log x. \qquad \square$$

Lemma 3.11 (asymp_log_ineq_growth_eventually)**.** *Let $\delta > 0$ and $a, b \in \mathbb{R}$. There is $x_0 \ge 1$ such that every $x \ge x_0$ satisfies $a \le \log x$ and $b \le x^\delta$.*

Lean: asymp_log_ineq_growth_eventually — proved by AIProver, job 2, session 19

```
theorem asymp_log_ineq_growth_eventually (δ a b : ℝ) (hδ : 0 < δ) :
    ∃ x₀ : ℝ, 1 ≤ x₀ ∧ ∀ x : ℝ, x₀ ≤ x → a ≤ Real.log x ∧ b ≤ x ^ δ := by
  have hlog : Filter.Tendsto Real.log Filter.atTop Filter.atTop := Real.tendsto_log_atTop
  have hrpow : Filter.Tendsto (fun x : ℝ => x ^ δ) Filter.atTop Filter.atTop :=
    tendsto_rpow_atTop hδ
  have hlog_event (C : ℝ) : ∃ x₀ : ℝ, ∀ x : ℝ, x₀ ≤ x → C ≤ Real.log x :=
    (Filter.tendsto_atTop_atTop.mp hlog) C
  have hrpow_event (C : ℝ) : ∃ x₀ : ℝ, ∀ x : ℝ, x₀ ≤ x → C ≤ x ^ δ :=
    (Filter.tendsto_atTop_atTop.mp hrpow) C
  rcases hlog_event a with ⟨xlog, hlogx⟩
  rcases hrpow_event b with ⟨xrpow, hrpowx⟩
  set x₀ := max (max xlog 1) xrpow with hx₀
  have hx₀_ge_1 : 1 ≤ x₀ := by
    rw [hx₀]
    exact le_trans (le_max_right xlog 1) (le_max_left _ _)
  have hxlog_le : xlog ≤ x₀ := by
    rw [hx₀]
    exact le_trans (le_max_left xlog 1) (le_max_left _ _)
  have hxrpow_le : xrpow ≤ x₀ := by
    rw [hx₀]
    exact le_max_right _ _
  refine ⟨x₀, hx₀_ge_1, λ x hx => ?_⟩
  have hxlogx : xlog ≤ x := le_trans hxlog_le hx
  have hxrpowx : xrpow ≤ x := le_trans hxrpow_le hx
  exact ⟨hlogx x hxlogx, hrpowx x hxrpowx⟩
```

Proof. Both $\log x$ and x^δ tend to $+\infty$. Take x_0 to be the maximum of 1 and the two thresholds. $\square$

Lemma 3.12 (asymp_log_ineq_growth)**.** *Let $\delta > 0$, $c \in \mathbb{R}$, $d > 0$. There is $x_0 \ge 1$ such that for all $x \ge x_0$,*

$$x^\delta \log\Big(\frac{x^{2\delta}\log(5x)}{\log 2} + 1\Big) + c\,x^{2\delta} < d\,x^{2\delta}\log x.$$

Lean: asymp_log_ineq_growth — proved by the captain after a hand-back

```
theorem asymp_log_ineq_growth (δ c d : ℝ) (hδ : 0 < δ) (hd : 0 < d) :
    ∃ x₀ : ℝ, 1 ≤ x₀ ∧ ∀ x : ℝ, x₀ ≤ x →
      x ^ δ * Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1) + c * x ^ (2 * δ)
        < d * x ^ (2 * δ) * Real.log x := by
  obtain ⟨x₀, hx₀1, hx₀⟩ := asymp_log_ineq_growth_eventually δ (max 1 (2 * (|c| + 1) / d))
    (2 * (Real.log 11 + 2 * δ + 1) / d) hδ
  refine ⟨x₀, hx₀1, fun x hx => ?_⟩
  obtain ⟨ha, hb⟩ := hx₀ x hx
  have hx1 : 1 ≤ x := le_trans hx₀1 hx
  have hxpos : 0 < x := by linarith
  have hu : 0 < x ^ δ := Real.rpow_pos_of_pos hxpos δ
```

```
  have hL1 : 1 ≤ Real.log x := le_trans (le_max_left _ _) ha
  have hdL : 2 * (|c| + 1) ≤ Real.log x * d := (div_le_iff₀ hd).mp (le_trans (le_max_right _ _) ha)
  have hdu : 2 * (Real.log 11 + 2 * δ + 1) ≤ x ^ δ * d := (div_le_iff₀ hd).mp hb
  have hlog11 : 0 < Real.log 11 := Real.log_pos (by norm_num)
  have e1 : 2 * (Real.log 11 + 2 * δ + 1) * Real.log x ≤ x ^ δ * d * Real.log x :=
    mul_le_mul_of_nonneg_right hdu (by linarith)
  have e2 : Real.log 11 ≤ Real.log 11 * Real.log x := by nlinarith
  have e3 : 2 * (|c| + 1) * x ^ δ ≤ Real.log x * d * x ^ δ := mul_le_mul_of_nonneg_right hdL hu.le
  have e4 : c * x ^ δ < (|c| + 1) * x ^ δ :=
    mul_lt_mul_of_pos_right (by linarith [le_abs_self c]) hu
  have key : Real.log 11 + (2 * δ + 1) * Real.log x + c * x ^ δ < d * x ^ δ * Real.log x := by
    nlinarith
  have key2 := mul_lt_mul_of_pos_left key hu
  have hlog := asymp_log_ineq_growth_logbound δ x hδ hx1
  have e5 : x ^ δ * Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1)
      ≤ x ^ δ * (Real.log 11 + (2 * δ + 1) * Real.log x) := mul_le_mul_of_nonneg_left hlog hu.le
  have hsq : x ^ (2 * δ) = x ^ δ * x ^ δ := by rw [two_mul, Real.rpow_add hxpos]
  rw [hsq] at e5 ⊢
  nlinarith [e5, key2]
```

Proof. Apply Lemma 3.11 with

$$a = \max(1,\, 2(|c|+1)/d), \qquad b = 2(\log 11 + 2\delta + 1)/d.$$

For $x \geq x_0$ put $u = x^\delta > 0$ and $L = \log x$. Then

$$L \geq 1, \qquad dL \geq 2(|c|+1), \qquad du \geq 2(\log 11 + 2\delta + 1).$$

Splitting duL into two halves gives

$$duL \;\geq\; (\log 11 + 2\delta + 1)L + (|c|+1)u \;>\; \log 11 + (2\delta+1)L + cu.$$

The last step uses $\log 11 \cdot L \geq \log 11$ and $(|c|+1)u > cu$.

Multiply by u and use $x^{2\delta} = u^2$. Lemma 3.10 bounds u times the logarithm by $u(\log 11 + (2\delta+1)L)$. Combining the two inequalities gives the claim. □

Lemma 3.13 (asymp_log_ineq_pointwise)**.** *Let $\delta, \varepsilon, x \in \mathbb{R}$ with $\delta > 0$, $4\delta < \varepsilon < 1$, and $x > 1$. Assume*

$$x^\delta \log\Big(\frac{x^{2\delta}\log(5x)}{\log 2} + 1\Big) + (1-\varepsilon)\log 5 \cdot x^{2\delta} < \frac{\delta}{2}\, x^{2\delta} \log x.$$

Let $k = \lfloor x^{2\delta} \rfloor_+$. Then $1 \leq k \leq x^{1-\delta}$ and

$$x^\delta \log\Big(\frac{k\log(5x)}{\log 2} + 1\Big) + (1-\varepsilon)k\log(5x) < k\log(x^{1-\delta}/k).$$

Lean: asymp_log_ineq_pointwise — proved by the captain after a hand-back

```
theorem asymp_log_ineq_pointwise (δ ε x : ℝ) (hδ : 0 < δ) (hδε : 4 * δ < ε) (hε1 : ε < 1)
    (hx : 1 < x)
    (hgrowth : x ^ δ * Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1)
        + (1 - ε) * Real.log 5 * x ^ (2 * δ) < δ / 2 * x ^ (2 * δ) * Real.log x) :
    1 ≤ (⌊x ^ (2 * δ)⌋₊ : ℝ) ∧ (⌊x ^ (2 * δ)⌋₊ : ℝ) ≤ x ^ (1 - δ) ∧
      x ^ δ * Real.log ((⌊x ^ (2 * δ)⌋₊ : ℝ) * Real.log (5 * x) / Real.log 2 + 1)
        + (1 - ε) * (⌊x ^ (2 * δ)⌋₊ : ℝ) * Real.log (5 * x)
```

```
        < (₊x ^ (2 * δ)₍₂₎ : ℝ) * Real.log (x ^ (1 - δ) / (₊x ^ (2 * δ)₍₂₎ : ℝ))
↪  := by
  have hxpos : 0 < x := by linarith
  have hx1 : 1 ≤ x := hx.le
  have hG : 0 < Real.log x := Real.log_pos hx
  have hl5 : 0 < Real.log 5 := Real.log_pos (by norm_num)
  have hl2 : 0 < Real.log 2 := Real.log_pos (by norm_num)
  have hL5 : Real.log (5 * x) = Real.log 5 + Real.log x :=
    Real.log_mul (by norm_num) hxpos.ne'
  have hL : 0 ≤ Real.log (5 * x) := by rw [hL5]; linarith
  have hy1 : 1 ≤ x ^ (2 * δ) := Real.one_le_rpow hx1 (by linarith)
  have hy0 : 0 ≤ x ^ (2 * δ) := by linarith
  have hyle : x ^ (2 * δ) ≤ x ^ (1 - δ) :=
    Real.rpow_le_rpow_of_exponent_le hx1 (by linarith)
  have hxd : 0 ≤ x ^ δ := Real.rpow_nonneg hxpos.le δ
  have hk1n : 1 ≤ ₊x ^ (2 * δ)₍₂₎ := Nat.le_floor (by exact_mod_cast hy1)
  have hk1 : (1:ℝ) ≤ (₊x ^ (2 * δ)₍₂₎ : ℝ) := by exact_mod_cast hk1n
  have hkle : (₊x ^ (2 * δ)₍₂₎ : ℝ) ≤ x ^ (2 * δ) := Nat.floor_le hy0
  have hkgt : x ^ (2 * δ) < (₊x ^ (2 * δ)₍₂₎ : ℝ) + 1 := Nat.lt_floor_add_one _
  set k : ℝ := (₊x ^ (2 * δ)₍₂₎ : ℝ) with hk
  have hk0 : 0 ≤ k := by linarith
  have hkpos : 0 < k := by linarith
  refine ⟨hk1, by linarith, ?_⟩
  have hc : 0 ≤ Real.log (5 * x) / Real.log 2 := div_nonneg hL hl2.le
  have hdiv : k * Real.log (5 * x) / Real.log 2
      ≤ x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 := by
    rw [mul_div_assoc, mul_div_assoc]
    exact mul_le_mul_of_nonneg_right hkle hc
  have hdiv0 : 0 ≤ k * Real.log (5 * x) / Real.log 2 := by
    rw [mul_div_assoc]
    exact mul_nonneg hk0 hc
  have hlogmono : Real.log (k * Real.log (5 * x) / Real.log 2 + 1)
      ≤ Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1) :=
    Real.log_le_log (by linarith) (by linarith)
  have hA := mul_le_mul_of_nonneg_left hlogmono hxd
  have hlogk : Real.log k ≤ 2 * δ * Real.log x := by
    have := Real.log_le_log hkpos hkle
    rwa [Real.log_rpow hxpos] at this
  have hR : Real.log (x ^ (1 - δ) / k) = (1 - δ) * Real.log x - Real.log k := by
    rw [Real.log_div (Real.rpow_pos_of_pos hxpos _).ne' hkpos.ne', Real.log_rpow hxpos]
  rw [hR]
  have h1 : (1 - ε) * Real.log 5 * k ≤ (1 - ε) * Real.log 5 * x ^ (2 * δ) :=
    mul_le_mul_of_nonneg_left hkle (mul_nonneg (by linarith) hl5.le)
  have h2 : k * Real.log k ≤ k * (2 * δ * Real.log x) :=
    mul_le_mul_of_nonneg_left hlogk hk0
  have h3 : δ / 2 * x ^ (2 * δ) * Real.log x ≤ δ / 2 * (k + 1) * Real.log x :=
    mul_le_mul_of_nonneg_right
      (mul_le_mul_of_nonneg_left hkgt.le (by linarith : (0:ℝ) ≤ δ / 2)) hG.le
  have h4 : 0 ≤ (k - 1) * Real.log x * δ :=
    mul_nonneg (mul_nonneg (by linarith) hG.le) hδ.le
  have h5 : 0 ≤ (ε - 4 * δ) * k * Real.log x :=
    mul_nonneg (mul_nonneg (by linarith) hk0) hG.le
  have h6 : (1 - ε) * k * Real.log (5 * x)
      = (1 - ε) * Real.log 5 * k + (1 - ε) * k * Real.log x := by
    rw [hL5]; ring
  nlinarith [hA, hgrowth, h1, h2, h3, h4, h5, h6]
```

Proof. Bounds on k. Since $x^{2\delta} \geq 1$ we get $k \geq 1$. Also $k \leq x^{2\delta} < k+1$. Since $3\delta < 1$, $x^{2\delta} \leq x^{1-\delta}$.

The two sides. Put $L = \log x > 0$. Monotonicity of log and $k \le x^{2\delta}$ bound the first term on the left by the corresponding term in the hypothesis. Moreover $\log k \le 2\delta L$, so

$$k \log(x^{1-\delta}/k) = k((1-\delta)L - \log k) \ge (1-3\delta)kL.$$

Using $\log(5x) = \log 5 + L$, the hypothesis, and $(1-\varepsilon)\log 5 \cdot k \le (1-\varepsilon)\log 5 \cdot x^{2\delta}$, the left side is

$$< \ \frac{\delta}{2} x^{2\delta} L + (1-\varepsilon)kL \ \le \ \frac{\delta}{2}(k+1)L + (1-\varepsilon)kL.$$

Comparison. It remains to check $\frac{\delta}{2}(k+1) + (1-\varepsilon)k \le (1-3\delta)k$. This holds because

$$(\varepsilon - 3\delta)k - \frac{\delta}{2}(k+1) = (\varepsilon - 4\delta)k + \frac{\delta}{2}(k-1) \ge 0. \qquad \square$$

Lemma 3.14 (asymp_log_ineq)**.** *Let $\delta > 0$ and $4\delta < \varepsilon < 1$. There is $x_0 \ge 1$ such that for all $x \ge x_0$, with $k = \lfloor x^{2\delta} \rfloor_+$, we have $1 \le k \le x^{1-\delta}$ and*

$$x^{\delta} \log\Big(\frac{k \log(5x)}{\log 2} + 1\Big) + (1-\varepsilon)k \log(5x) < k \log(x^{1-\delta}/k).$$

Lean: asymp_log_ineq — proved by the captain after a hand-back

```
theorem asymp_log_ineq (δ ε : ℝ) (hδ : 0 < δ) (hδε : 4 * δ < ε) (hε1 : ε < 1) :
    ∃ x₀ : ℝ, 1 ≤ x₀ ∧ ∀ x : ℝ, x₀ ≤ x →
      1 ≤ (x ^ (2 * δ) : ℝ) ∧ (x ^ (2 * δ) : ℝ) ≤ x ^ (1 - δ) ∧
      x ^ δ * Real.log ((x ^ (2 * δ) : ℝ) * Real.log (5 * x) / Real.log 2 + 1)
        + (1 - ε) * (x ^ (2 * δ) : ℝ) * Real.log (5 * x)
        < (x ^ (2 * δ) : ℝ) * Real.log (x ^ (1 - δ) / (x ^ (2 * δ) : ℝ))
↪   := by
  obtain ⟨x₀, hx₀1, h⟩ := asymp_log_ineq_growth δ ((1 - ε) * Real.log 5) (δ / 2) hδ (by linarith)
  refine ⟨x₀ + 1, by linarith, fun x hx => ?_⟩
  exact asymp_log_ineq_pointwise δ ε x hδ hδε hε1 (by linarith) (h x (by linarith))
```

Proof. Take x_0' from Lemma 3.12 with $c = (1-\varepsilon)\log 5$ and $d = \delta/2$, and set $x_0 = x_0' + 1$. Then every $x \ge x_0$ satisfies $x > 1$ and the hypothesis of Lemma 3.13, which gives the conclusion. $\square$

Lemma 3.15 (asymp_main_lhs_log)**.** *Let $x, \delta, \varepsilon \in \mathbb{R}$ and $k \in \mathbb{N}$ with $x \ge 1$ and $\varepsilon < 1$. Put $N = \lfloor 5x \rfloor_+$ and*

$$A = (\lg(N^k) + 1)^{\lfloor x^{\delta} \rfloor_+} \cdot (N^k)^{1-\varepsilon}.$$

Then $A > 0$ and

$$\log A \le x^{\delta} \log\Big(\frac{k \log(5x)}{\log 2} + 1\Big) + (1-\varepsilon)k \log(5x).$$

Lean: asymp_main_lhs_log — proved by AIProver, job 3, session 0

```
theorem asymp_main_lhs_log (x δ ε : ℝ) (k : ℕ) (hx : 1 ≤ x) (hε1 : ε < 1) :
    0 < ((Nat.log 2 (5 * x ^ k) + 1 : ℕ) : ℝ) ^ x ^ δ
        * ((5 * x ^ k : ℕ) : ℝ) ^ (1 - ε) ∧
    Real.log (((Nat.log 2 (5 * x ^ k) + 1 : ℕ) : ℝ) ^ x ^ δ
        * ((5 * x ^ k : ℕ) : ℝ) ^ (1 - ε))
      ≤ x ^ δ * Real.log ((k : ℝ) * Real.log (5 * x) / Real.log 2 + 1)
        + (1 - ε) * (k : ℝ) * Real.log (5 * x) := by
```

```
  have hx5pos : 0 < (5 : ℝ) * x := by nlinarith
  have hfloor5x_pos_nat : 0 < (⌊(5 : ℝ) * x⌋₊ : ℕ) := Nat.floor_pos.mpr (by nlinarith)
  have hfloor5x_pos : 0 < ((⌊(5 : ℝ) * x⌋₊ : ℕ) : ℝ) := by exact_mod_cast hfloor5x_pos_nat
  have hfloor5x_le : (⌊(5 : ℝ) * x⌋₊ : ℝ) ≤ (5 : ℝ) * x := Nat.floor_le hx5pos.le
  have h_first_pos : 0 < ((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 : ℕ) : ℝ) ^ ⌊x ^ δ⌋₊ :=
↪   by
    positivity
  have hpos_base_B : 0 < ((⌊5 * x⌋₊ ^ k : ℕ) : ℝ) := by
    exact_mod_cast pow_pos hfloor5x_pos_nat k
  have h_second_pos : 0 < ((⌊5 * x⌋₊ ^ k : ℕ) : ℝ) ^ (1 - ε) :=
    Real.rpow_pos_of_pos hpos_base_B (1 - ε)
  constructor
  · exact mul_pos h_first_pos h_second_pos
  ·
    have hpos_log2 : 0 < Real.log 2 := Real.log_pos (by norm_num : (1 : ℝ) < 2)
    have hpos_log5x : 0 < Real.log ((5 : ℝ) * x) := Real.log_pos (by nlinarith)




    have hA_pos : 0 < ((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 : ℕ) : ℝ) := by positivity
    have h_split : Real.log (((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 : ℕ) : ℝ) ^ ⌊x ^ δ⌋₊
        * ((⌊5 * x⌋₊ ^ k : ℕ) : ℝ) ^ (1 - ε)) =
        Real.log (((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 : ℕ) : ℝ) ^ ⌊x ^ δ⌋₊)
        + Real.log (((⌊5 * x⌋₊ ^ k : ℕ) : ℝ) ^ (1 - ε)) := by
      rw [Real.log_mul (by
        have : ((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 : ℕ) : ℝ) ^ ⌊x ^ δ⌋₊ ≠ 0 := by
↪   positivity
        exact this)
        (h_second_pos.ne.symm)]
    rw [h_split]

    have h_extract : Real.log (((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 : ℕ) : ℝ) ^ ⌊x ^
↪   δ⌋₊)
        + Real.log (((⌊5 * x⌋₊ ^ k : ℕ) : ℝ) ^ (1 - ε)) =
        ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 : ℕ) :
↪   ℝ))
        + (1 - ε) * Real.log (((⌊5 * x⌋₊ ^ k : ℕ) : ℝ)) := by
      calc
        Real.log (((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 : ℕ) : ℝ) ^ ⌊x ^ δ⌋₊)
            + Real.log (((⌊5 * x⌋₊ ^ k : ℕ) : ℝ) ^ (1 - ε))
        = Real.log (((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 : ℕ) : ℝ) ^ ((⌊x ^ δ⌋₊ : ℕ)
↪   : ℝ))
            + Real.log (((⌊5 * x⌋₊ ^ k : ℕ) : ℝ) ^ (1 - ε)) := by
          simp
        _ = ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 :
↪   ℕ) : ℝ))
            + Real.log (((⌊5 * x⌋₊ ^ k : ℕ) : ℝ) ^ (1 - ε)) := by
          rw [Real.log_rpow hA_pos ((⌊x ^ δ⌋₊ : ℕ) : ℝ)]
        _ = ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 :
↪   ℕ) : ℝ))
            + (1 - ε) * Real.log (((⌊5 * x⌋₊ ^ k : ℕ) : ℝ)) := by
          rw [Real.log_rpow hpos_base_B (1 - ε)]
        _ = ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 :
↪   ℕ) : ℝ))
            + (1 - ε) * Real.log (((⌊5 * x⌋₊ ^ k : ℕ) : ℝ)) := rfl
    rw [h_extract]
```

```
    have h_floor_le : ((⌊x ^ δ⌋₊ : ℕ) : ℝ) ≤ x ^ δ := by
      have h_nonneg : 0 ≤ x ^ δ := Real.rpow_nonneg (by linarith) δ
      exact Nat.floor_le h_nonneg
    have h_one_minus_eps_nonneg : 0 ≤ 1 - ε := by nlinarith
    have h_log_arg_nonneg : 0 ≤ Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2) + 1) :=
      Real.log_nonneg (by
        have h_nonneg_div : 0 ≤ (k : ℝ) * Real.log (5 * x) / Real.log 2 :=
          div_nonneg (mul_nonneg (Nat.cast_nonneg _) hpos_log5x.le) hpos_log2.le
        nlinarith)

    have h_log_bound : (Nat.log 2 (⌊(5 : ℝ) * x⌋₊ ^ k) : ℝ) ≤ (k : ℝ) * Real.log ((5 : ℝ) *
↪  x) / Real.log 2 := by
      let n := ⌊(5 : ℝ) * x⌋₊ ^ k
      have hn_pos : n ≠ 0 := pow_ne_zero k hfloor5x_pos_nat.ne'
      have h_pow := Nat.pow_log_le_self 2 hn_pos
      have h_pow' : ((2 : ℕ) ^ Nat.log 2 n : ℝ) ≤ (n : ℝ) := by exact_mod_cast h_pow
      have h_pow'' : ((2 : ℝ) ^ (Nat.log 2 n : ℝ)) ≤ (n : ℝ) := by
        simpa [Real.rpow_natCast] using h_pow'
      have h_log_pow : Real.log ((2 : ℝ) ^ (Nat.log 2 n : ℝ)) ≤ Real.log (n : ℝ) :=
        Real.log_le_log (by positivity) h_pow''
      rw [Real.log_rpow (by norm_num : (0 : ℝ) < 2) (Nat.log 2 n : ℝ)] at h_log_pow
      have h_log_n : Real.log (n : ℝ) = (k : ℝ) * Real.log ((⌊(5 : ℝ) * x⌋₊ : ℕ) : ℝ) := by
        simp [n, Real.log_pow]
      rw [h_log_n] at h_log_pow
      have h_div : (Nat.log 2 n : ℝ) ≤ (k : ℝ) * Real.log ((⌊(5 : ℝ) * x⌋₊ : ℕ) : ℝ) /
↪  Real.log 2 := by
        rw [le_div_iff₀ hpos_log2]
        nlinarith
      have h_log_floor_le : Real.log ((⌊(5 : ℝ) * x⌋₊ : ℕ) : ℝ) ≤ Real.log ((5 : ℝ) * x) :=
        Real.log_le_log hfloor5x_pos hfloor5x_le
      calc
        (Nat.log 2 n : ℝ) ≤ (k : ℝ) * Real.log ((⌊(5 : ℝ) * x⌋₊ : ℕ) : ℝ) / Real.log 2 :=
↪  h_div
        _ ≤ (k : ℝ) * Real.log ((5 : ℝ) * x) / Real.log 2 := by
          gcongr

    have h_bound_step3 : ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((Nat.log 2 (⌊5 * x⌋₊
↪  ^ k) + 1 : ℕ) : ℝ))
        + (1 - ε) * Real.log ((((⌊5 * x⌋₊ ^ k : ℕ) : ℝ))
        ≤ ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2) + 1)
          + (1 - ε) * Real.log ((((⌊5 * x⌋₊ ^ k : ℕ) : ℝ)) := by
      refine add_le_add ?_ (le_refl _)
      have h_coeff_nonneg : 0 ≤ ((⌊x ^ δ⌋₊ : ℕ) : ℝ) := Nat.cast_nonneg _
      refine mul_le_mul_of_nonneg_left ?_ h_coeff_nonneg
      refine Real.log_le_log (by positivity) ?_

      push_cast

      gcongr


    have h_log_B : Real.log ((((⌊5 * x⌋₊ ^ k : ℕ) : ℝ)) = (k : ℝ) * Real.log ((((⌊5 *
↪  x⌋₊ : ℕ) : ℝ)) := by
      simp [Real.log_pow]
    have h_log_B_bound : (1 - ε) * Real.log ((((⌊5 * x⌋₊ ^ k : ℕ) : ℝ))
        ≤ (1 - ε) * ((k : ℝ) * Real.log ((5 : ℝ) * x)) := by
```

```
      rw [h_log_B]
      refine mul_le_mul_of_nonneg_left ?_ h_one_minus_eps_nonneg
      refine mul_le_mul_of_nonneg_left (Real.log_le_log hfloor5x_pos hfloor5x_le) (by exact_mod_cast
↪  Nat.zero_le k)




    have h_final : ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2)
↪  + 1)
        + (1 - ε) * ((k : ℝ) * Real.log ((5 : ℝ) * x))
        ≤ x ^ δ * Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2) + 1)
          + (1 - ε) * (k : ℝ) * Real.log (5 * x) := by
      refine add_le_add (mul_le_mul_of_nonneg_right h_floor_le h_log_arg_nonneg) ?_
      ring_nf
      rfl
    calc
      ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 : ℕ) : ℝ))
          + (1 - ε) * Real.log ((((⌊5 * x⌋₊ ^ k : ℕ) : ℝ))
      ≤ ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2) + 1)
          + (1 - ε) * Real.log ((((⌊5 * x⌋₊ ^ k : ℕ) : ℝ)) := h_bound_step3
      _ ≤ ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2) + 1)
          + (1 - ε) * ((k : ℝ) * Real.log ((5 : ℝ) * x)) := by
        refine add_le_add (le_refl _) h_log_B_bound
      _ ≤ x ^ δ * Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2) + 1)
          + (1 - ε) * (k : ℝ) * Real.log (5 * x) := h_final
```

Proof. Since $5x \geq 5$ we have $N \geq 1$, so $A > 0$, and

$$\log A = \lfloor x^{\delta} \rfloor_{+} \log(\lg(N^k) + 1) + (1 - \varepsilon) k \log N.$$

- From $2^{\lg(N^k)} \leq N^k$ we get $\lg(N^k) \leq k \log N / \log 2 \leq k \log(5x) / \log 2$, using $N \leq 5x$.
- $0 \leq \lfloor x^{\delta} \rfloor_{+} \leq x^{\delta}$, and the logarithm $\log\bigl(k \log(5x)/\log 2 + 1\bigr)$ is nonnegative.
- $\log N \leq \log(5x)$ and $1 - \varepsilon \geq 0$.

Substituting these bounds term by term gives the claim. □

Lemma 3.16 (asymp_main_rhs_log). *Let $x, \delta \in \mathbb{R}$ and $k, M \in \mathbb{N}$ with $1 \leq k$, $k \leq x^{1-\delta}$ and $x^{1-\delta} \leq M$. Then*

$$k \log(x^{1-\delta}/k) \leq \log \binom{M}{k}.$$

Lean: asymp_main_rhs_log — proved by AIProver, job 3, session 0

```
theorem asymp_main_rhs_log (x δ : ℝ) (k M : ℕ) (hk1 : 1 ≤ k)
    (hkx : (k : ℝ) ≤ x ^ (1 - δ)) (hM : x ^ (1 - δ) ≤ (M : ℝ)) :
    (k : ℝ) * Real.log (x ^ (1 - δ) / (k : ℝ)) ≤ Real.log (M.choose k : ℝ) := by
  have hkpos : 0 < (k : ℝ) := by
    have := Nat.one_pos.trans_le hk1
    exact_mod_cast this
  have h_nonzero_k : (k : ℝ) ≠ 0 := by linarith
  have hk_le_M_real : (k : ℝ) ≤ (M : ℝ) := by
    linarith
```

```
  have hk_le_M : (k : ℕ) ≤ M := by
    exact_mod_cast hk_le_M_real
  have hM_gt_0 : 0 < (M : ℝ) := by
    have : 1 ≤ (M : ℝ) := by
      calc
        1 ≤ (k : ℝ) := by exact_mod_cast hk1
        _ ≤ (M : ℝ) := hk_le_M_real
    linarith
  have hx_pow_pos : 0 < x ^ (1 - δ) := lt_of_lt_of_le hkpos hkx
  have hM_div_k_ge : x ^ (1 - δ) / (k : ℝ) ≤ (M : ℝ) / (k : ℝ) :=
    div_le_div_of_nonneg_right hM (by linarith)
  have h_div_ge_one : 1 ≤ x ^ (1 - δ) / (k : ℝ) := by
    calc
      1 = (k : ℝ) / (k : ℝ) := by field_simp [h_nonzero_k]
      _ ≤ x ^ (1 - δ) / (k : ℝ) := div_le_div_of_nonneg_right hkx (by linarith)
  have h_log_ineq : Real.log (x ^ (1 - δ) / (k : ℝ)) ≤ Real.log ((M : ℝ) / (k : ℝ)) :=
    Real.log_le_log (div_pos hx_pow_pos hkpos) hM_div_k_ge
  have h_choose_pos_nat : 0 < M.choose k :=
    Nat.choose_pos hk_le_M
  have h_choose_ge_pow_nat : ((M : ℝ) / (k : ℝ)) ^ k ≤ (M.choose k : ℝ) :=
    choose_ge_pow_div M k hk_le_M
  have h_choose_pos : 0 < (M.choose k : ℝ) := by exact_mod_cast h_choose_pos_nat
  have h_div_pos : 0 < ((M : ℝ) / (k : ℝ)) := div_pos hM_gt_0 hkpos
  have h_choose_ge_pow : ((M : ℝ) / (k : ℝ)) ^ (k : ℝ) ≤ (M.choose k : ℝ) := by
    have : ((M : ℝ) / (k : ℝ)) ^ (k : ℝ) = ((M : ℝ) / (k : ℝ)) ^ k := by norm_cast
    rw [this]
    exact h_choose_ge_pow_nat
  have h_log_choose_ge : Real.log (((M : ℝ) / (k : ℝ)) ^ (k : ℝ)) ≤ Real.log (M.choose k : ℝ) :=
    Real.log_le_log (Real.rpow_pos_of_pos h_div_pos (k : ℝ)) h_choose_ge_pow
  calc
    (k : ℝ) * Real.log (x ^ (1 - δ) / (k : ℝ)) ≤ (k : ℝ) * Real.log ((M : ℝ) / (k : ℝ)) := by
      nlinarith
    _ = Real.log (((M : ℝ) / (k : ℝ)) ^ (k : ℝ)) := by
      rw [Real.log_rpow h_div_pos (k : ℝ)]
    _ ≤ Real.log (M.choose k : ℝ) := h_log_choose_ge
```

Proof. We have $k \le M$, $0 < x^{1-\delta}/k \le M/k$, and $\binom{M}{k} > 0$. Hence, using Lemma 3.9 in the last step,

$$k \log(x^{1-\delta}/k) \le k \log(M/k) = \log((M/k)^k) \le \log \binom{M}{k}. \qquad \square$$

Lemma 3.17 (asymp_main)**.** *Let $\delta > 0$ and $4\delta < \varepsilon < 1$. There is $x_0 \ge 1$ with the following property. Let $x \ge x_0$, put $k = \lfloor x^{2\delta} \rfloor_+$ and $N = \lfloor 5x \rfloor_+$, and let $M \in \mathbb{N}$ with $x^{1-\delta} \le M$. Then $1 \le k \le M$ and*

$$(\lg(N^k) + 1)^{\lfloor x^{\delta} \rfloor_+} (N^k)^{1-\varepsilon} < \binom{M}{k}.$$

Lean: asymp_main — proved by the captain after a hand-back

```
theorem asymp_main (δ ε : ℝ) (hδ : 0 < δ) (hδε : 4 * δ < ε) (hε1 : ε < 1) :
    ∃ x₀ : ℝ, 1 ≤ x₀ ∧ ∀ x : ℝ, x₀ ≤ x → ∀ M : ℕ, x ^ (1 - δ) ≤ (M : ℝ) →
      1 ≤ U+230Ax ^ (2 * δ)U+230BU+208A ∧ U+230Ax ^ (2 * δ)U+230BU+208A ≤ M ∧
      ((Nat.log 2 (U+230A5 * xU+230BU+208A ^ U+230Ax ^ (2 * δ)U+230BU+208A) + 1 : ℕ) : ℝ) ^ U+230Ax ^
↪   δU+230BU+208A
        * ((U+230A5 * xU+230BU+208A ^ U+230Ax ^ (2 * δ)U+230BU+208A : ℕ) : ℝ) ^ (1 - ε)
```

```
      < (M.choose U+230Ax ^ (2 * δ)U+230BU+208A : ℝ) := by
  obtain ⟨x₀, hx₀1, hx₀⟩ := asymp_log_ineq δ ε hδ hδε hε1
  refine ⟨x₀, hx₀1, fun x hx M hM => ?_⟩
  obtain ⟨hk1, hkx, hlog⟩ := hx₀ x hx
  have hk1' : 1 ≤ U+230Ax ^ (2 * δ)U+230BU+208A := by exact_mod_cast hk1
  have hkM : U+230Ax ^ (2 * δ)U+230BU+208A ≤ M := by exact_mod_cast hkx.trans hM
  refine ⟨hk1', hkM, ?_⟩
  obtain ⟨hpos, hL⟩ := asymp_main_lhs_log x δ ε U+230Ax ^ (2 * δ)U+230BU+208A (by linarith) hε1
  have hR := asymp_main_rhs_log x δ U+230Ax ^ (2 * δ)U+230BU+208A M hk1' hkx hM
  have hCpos : (0 : ℝ) < (M.choose U+230Ax ^ (2 * δ)U+230BU+208A : ℝ) :=
    Nat.cast_pos.mpr (Nat.choose_pos hkM)
  rw [← Real.log_lt_log_iff hpos hCpos]
  linarith
```

Proof. Take x_0 from Lemma 3.14. That lemma gives $1 \le k \le x^{1-\delta} \le M$. Both sides of the desired inequality are positive, the left by Lemma 3.15 and the right because $k \le M$. So it suffices to compare logarithms. Chain three inequalities:

- the upper bound of Lemma 3.15 for the logarithm of the left side;
- the strict inequality of Lemma 3.14;
- the lower bound of Lemma 3.16 for $\log \binom{M}{k}$.

This is the formal version of comparing (1) and (2). □

Lemma 3.18 (`exists_prime_finset`). *Assume (H). Let $\delta, \eta \in \mathbb{R}$ with $0 < \delta < 1/4$ and $\eta > 0$. There is x_0 such that for every $x \ge x_0$ there is a finite set $S \subset \mathbb{N}$ with $|S| \ge x^{1-\eta}$ in which every p satisfies:*

- *p is prime;*
- *$2x < p \le 5x$;*
- *$p - 1$ is x^{δ}-smooth.*

Lean: `exists_prime_finset` — proved by AIProver, job 2, session 0

```
theorem exists_prime_finset (hH : SmoothShiftedPrimesHypothesis) (δ η : ℝ)
    (hδ : 0 < δ) (hδ4 : δ < 1 / 4) (hη : 0 < η) :
    ∃ x₀ : ℝ, ∀ x : ℝ, x₀ ≤ x → ∃ S : Finset ℕ, x ^ (1 - η) ≤ (S.card : ℝ) ∧
      ∀ p ∈ S, p.Prime ∧ 2 * x < (p : ℝ) ∧ (p : ℝ) ≤ 5 * x ∧
        IsSmoothNat (p - 1) (x ^ δ) := by
  have hη2 : 0 < η / 2 := half_pos hη
  rcases hH δ hδ hδ4 (η / 2) hη2 with ⟨x₀, hH'⟩
  let x₁ : ℝ := max x₀ 1
  have hx₁1 : 1 ≤ x₁ := le_max_right _ _
  have hx₁0 : x₀ ≤ x₁ := le_max_left _ _
  refine ⟨x₁, λ x hx => ?_⟩
  have hx1 : 1 ≤ x := hx₁1.trans hx
  have hx0 : x₀ ≤ x := hx₁0.trans hx
  have hcard := hH' x hx0

  let A : Set ℕ := {p | p.Prime ∧ 2 * x < (p : ℝ) ∧ (p : ℝ) ≤ 5 * x ∧ IsSmoothNat (p - 1) (x ^ δ)}
  have hA_finite : A.Finite := by
    have h_bound : A ⊆ {p | (p : ℝ) ≤ 5 * x} := by
      intro p ⟨_, _, hp5, _⟩; exact hp5
```

```
    have h_finite_Iic : {p : ℕ | (p : ℝ) ≤ 5 * x}.Finite := by
      have h_eq : {p : ℕ | (p : ℝ) ≤ 5 * x} = Set.Iic (⌊5 * x⌋₊ : ℕ) := by
        ext n
        simp [Set.mem_Iic]
        constructor
        · intro h
          have hpos : 0 ≤ 5 * x := by linarith
          exact (Nat.le_floor_iff hpos).mpr h
        · intro h
          have hpos : 0 ≤ 5 * x := by linarith
          have h_n_le_floor : (n : ℝ) ≤ (⌊5 * x⌋₊ : ℝ) := by exact mod_cast h
          have h_floor_le : (⌊5 * x⌋₊ : ℝ) ≤ 5 * x := Nat.floor_le hpos
          exact h_n_le_floor.trans h_floor_le
      rw [h_eq]
      exact Set.finite_Iic _
    exact h_finite_Iic.subset h_bound

  let S := hA_finite.toFinset
  have hS_card_nat : S.card = Nat.card A := by
    haveI : Fintype A := hA_finite.fintype
    simpa [S] using hA_finite.card_toFinset
  have hcard_nat : (Nat.card A : ℝ) ≥ x ^ (1 - η / 2) := by
    simpa [A] using hcard
  have hcard_S : (S.card : ℝ) ≥ x ^ (1 - η / 2) := by
    rw [hS_card_nat]; exact hcard_nat

  have h_exp_mono : (1 : ℝ) - η ≤ (1 : ℝ) - η / 2 := by linarith
  have h_pow_mono : x ^ (1 - η) ≤ x ^ (1 - η / 2) :=
    Real.rpow_le_rpow_of_exponent_le hx1 h_exp_mono
  have hgoal : x ^ (1 - η) ≤ (S.card : ℝ) := by
    linarith
  have hS_prop : ∀ p ∈ S, p.Prime ∧ 2 * x < (p : ℝ) ∧ (p : ℝ) ≤ 5 * x ∧ IsSmoothNat (p - 1) (x ^ δ) := by
    intro p hp
    have hpA : p ∈ A := by
      simpa [S] using hp
    rcases hpA with ⟨hp_prime, hp_low, hp_high, hp_smooth⟩
    exact ⟨hp_prime, hp_low, hp_high, hp_smooth⟩
  exact ⟨S, hgoal, hS_prop⟩
```

Proof. Apply (H) with δ and $\eta/2$ to obtain x_0', and set $x_0 = \max(x_0', 1)$. For $x \geq x_0$, the set A of primes counted in (H) is contained in $\{0, \ldots, \lfloor 5x \rfloor_+\}$, so it is finite. Take $S = A$. Then

$$|S| \geq x^{1-\eta/2} \geq x^{1-\eta},$$

because $x \geq 1$. □

Lemma 3.19 (good_n_from_finset)**.** *Let $S \subset \mathbb{N}$ be finite, $x, \delta, \varepsilon \in \mathbb{R}$, and $k \in \mathbb{N}$, with $x \geq 1$ and $\varepsilon < 1$. Suppose every $p \in S$ is prime, satisfies $2x < p \leq 5x$, and has $p - 1$ being x^{δ}-smooth. Suppose also $1 \leq k \leq |S|$. Put $N = \lfloor 5x \rfloor_+$ and assume*

$$(\lg(N^k) + 1)^{\lfloor x^{\delta} \rfloor_+} (N^k)^{1-\varepsilon} < \binom{|S|}{k}.$$

Then there is $n \in \mathbb{N}$ with $x < n$, $0 < n$ and $g(n) > n^{1-\varepsilon}$.

Lean: good_n_from_finset — proved by the captain after a hand-back

```
theorem good_n_from_finset (S : Finset ℕ) (x δ ε : ℝ) (k : ℕ) (hx : 1 ≤ x)
    (hε1 : ε < 1)
    (hS : ∀ p ∈ S, p.Prime ∧ 2 * x < (p : ℝ) ∧ (p : ℝ) ≤ 5 * x ∧
        IsSmoothNat (p - 1) (x ^ δ))
    (hk1 : 1 ≤ k) (hkS : k ≤ S.card)
    (hineq : ((Nat.log 2 (⁅U+230A⁆5 * x⁅U+230B⁆⁅U+208A⁆ ^ k) + 1 : ℕ) : ℝ) ^ ⁅U+230A⁆x ^ δ⁅U+230B⁆⁅U+208A⁆
        * ((⁅U+230A⁆5 * x⁅U+230B⁆⁅U+208A⁆ ^ k : ℕ) : ℝ) ^ (1 - ε) < (S.card.choose k : ℝ)) :
    ∃ n : ℕ, x < (n : ℝ) ∧ 0 < n ∧
      ((totientFiberCard n : ℕ) : ℝ) > (n : ℝ) ^ (1 - ε) := by
  obtain ⟨N, hN⟩ : ∃ N : ℕ, N = ⁅U+230A⁆5 * x⁅U+230B⁆⁅U+208A⁆ := ⟨_, rfl⟩
  rw [← hN] at hineq
  have hSp : ∀ p ∈ S, 0 < p - 1 ∧ p - 1 ≤ N ∧ IsSmoothNat (p - 1) (x ^ δ) := by
    intro p hp
    obtain ⟨hpr, h2x, h5x, hsm⟩ := hS p hp
    have h2 := hpr.two_le
    have hpN : p ≤ N := by
      rw [hN]; exact Nat.le_floor h5x
    exact ⟨by omega, by omega, hsm⟩
  obtain ⟨V, hVcard, hVmem⟩ := smooth_count (N ^ k) (x ^ δ)
  have hV : ∀ Q ⊆ S, Q.card = k → ∏ p ∈ Q, (p - 1) ∈ V := by
    intro Q hQ hQk
    have h := prod_smooth_bounded Q (x ^ δ) N (fun p hp => hSp p (hQ hp))
    rw [hQk] at h
    exact hVmem _ h.1 h.2.1 h.2.2
  obtain ⟨Q, hQS, hQk, hchoose⟩ :=
    exists_large_fiber S (fun p hp => (hS p hp).1) k hkS V hV
  have hprod := prod_smooth_bounded Q (x ^ δ) N (fun p hp => hSp p (hQS hp))
  rw [hQk] at hprod
  obtain ⟨hpos, hle, -⟩ := hprod
  refine ⟨∏ p ∈ Q, (p - 1), ?_, hpos, ?_⟩
  · have hne : Q.Nonempty := Finset.card_pos.mp (by omega)
    obtain ⟨p0, hp0⟩ := hne
    obtain ⟨hpr, h2x, -, -⟩ := hS p0 (hQS hp0)
    have h1 : p0 - 1 ≤ ∏ p ∈ Q, (p - 1) :=
      Nat.le_of_dvd hpos (Finset.dvd_prod_of_mem (fun p => p - 1) hp0)
    have h2 : (x : ℝ) < ((p0 - 1 : ℕ) : ℝ) := by
      have h1p : 1 ≤ p0 := hpr.one_lt.le
      rw [Nat.cast_sub h1p]
      push_cast
      linarith
    exact lt_of_lt_of_le h2 (Nat.cast_le.mpr h1)
  · have hVR : (V.card : ℝ) ≤ ((Nat.log 2 (N ^ k) + 1 : ℕ) : ℝ) ^ ⁅U+230A⁆x ^ δ⁅U+230B⁆⁅U+208A⁆ := by
      exact_mod_cast hVcard
    have hnP : ((∏ p ∈ Q, (p - 1) : ℕ) : ℝ) ^ (1 - ε) ≤ ((N ^ k : ℕ) : ℝ) ^ (1 - ε) :=
      Real.rpow_le_rpow (Nat.cast_nonneg _) (Nat.cast_le.mpr hle) (by linarith)
    have hcR : (S.card.choose k : ℝ) ≤
        (V.card : ℝ) * ((totientFiberCard (∏ p ∈ Q, (p - 1)) : ℕ) : ℝ) := by
      have h := (Nat.cast_le (α := ℝ)).mpr hchoose
      rwa [Nat.cast_mul] at h
    by_contra hcon
    push_neg at hcon
    have h1 := mul_le_mul_of_nonneg_left hcon (Nat.cast_nonneg (α := ℝ) V.card)
    have h2 := mul_le_mul_of_nonneg_left hnP (Nat.cast_nonneg (α := ℝ) V.card)
    have h3 := mul_le_mul_of_nonneg_right hVR
      (Real.rpow_nonneg (Nat.cast_nonneg (α := ℝ) (N ^ k)) (1 - ε))
    have := lt_of_le_of_lt (h1.trans (h2.trans h3)) (lt_of_lt_of_le hineq hcR)
    exact lt_irrefl _ this
```

Proof. Setting up the pigeonhole. Each $p \in S$ satisfies $p \geq 2$ and $p \leq N$, since p is an integer $\leq 5x$. Thus $0 < p - 1 \leq N$, and $p - 1$ is x^δ-smooth.

Let V be the set given by Lemma 3.6 for N^k and $y = x^\delta$. By Lemma 3.7, every k-subset $Q \subseteq S$ has $\prod_{p \in Q}(p-1)$ positive, at most N^k, and x^δ-smooth, so this product lies in V.

Applying the pigeonhole. Lemma 3.5 gives $Q \subseteq S$ with $|Q| = k$ and $\binom{|S|}{k} \leq |V|\, g(n)$, where $n = \prod_{p \in Q}(p-1)$. By Lemma 3.7, $0 < n \leq N^k$.

The bound $n > x$. Since $k \geq 1$, Q contains some p_0. Then $p_0 - 1 \mid n$, so

$$n \geq p_0 - 1 > 2x - 1 \geq x.$$

The bound on $g(n)$. Suppose $g(n) \leq n^{1-\varepsilon}$. Since $1 - \varepsilon \geq 0$ we have $n^{1-\varepsilon} \leq (N^k)^{1-\varepsilon}$. Also $|V| \leq (\lg(N^k) + 1)^{\lfloor x^\delta \rfloor_+}$. Then

$$|V|\, g(n) \leq |V|\, (N^k)^{1-\varepsilon} \leq (\lg(N^k) + 1)^{\lfloor x^\delta \rfloor_+} (N^k)^{1-\varepsilon} < \binom{|S|}{k} \leq |V|\, g(n),$$

a contradiction. □

Lemma 3.20 (`exists_good_n_large`)**.** *Assume (H). Let $0 < \varepsilon < 1$ and $C \in \mathbb{R}$. There is $n \in \mathbb{N}$ with $C < n$, $0 < n$ and $g(n) > n^{1-\varepsilon}$.*

Lean: `exists_good_n_large` — proved by the captain after a hand-back

```
theorem exists_good_n_large (hH : SmoothShiftedPrimesHypothesis) (ε : ℝ)
    (hε : 0 < ε) (hε1 : ε < 1) (C : ℝ) :
    ∃ n : ℕ, C < (n : ℝ) ∧ 0 < n ∧
      ((totientFiberCard n : ℕ) : ℝ) > (n : ℝ) ^ (1 - ε) := by
  obtain ⟨x₀, hx₀⟩ := exists_prime_finset hH (ε / 8) (ε / 8) (by linarith) (by linarith) (by linarith)
  obtain ⟨x₁, _hx₁1, hx₁⟩ := asymp_main (ε / 8) ε (by linarith) (by linarith) hε1
  obtain ⟨x, hx0, hx1, hxC, hx1'⟩ : ∃ x : ℝ, x₀ ≤ x ∧ x₁ ≤ x ∧ C ≤ x ∧ 1 ≤ x :=
    ⟨max (max x₀ x₁) (max C 1),
      le_trans (le_max_left _ _) (le_max_left _ _),
      le_trans (le_max_right _ _) (le_max_left _ _),
      le_trans (le_max_left _ _) (le_max_right _ _),
      le_trans (le_max_right _ _) (le_max_right _ _)⟩
  obtain ⟨S, hScard, hS⟩ := hx₀ x hx0
  obtain ⟨hk1, hkS, hineq⟩ := hx₁ x hx1 S.card hScard
  obtain ⟨n, hxn, hn0, hn⟩ :=
    good_n_from_finset S x (ε / 8) ε x ^ (2 * (ε / 8)) hx1' hε1 hS hk1 hkS hineq
  exact ⟨n, lt_of_le_of_lt hxC hxn, hn0, hn⟩
```

Proof. Set $\delta = \varepsilon/8$. Then $0 < \delta < 1/4$ and $4\delta < \varepsilon$.

Let x_0 come from Lemma 3.18 with $\eta = \delta$, and x_1 from Lemma 3.17. Choose

$$x \geq \max(x_0, x_1, C, 1).$$

Lemma 3.18 provides S with $|S| \geq x^{1-\delta}$. Lemma 3.17 with $M = |S|$ and $k = \lfloor x^{2\delta} \rfloor_+$ gives $1 \leq k \leq |S|$ and the inequality required by Lemma 3.19. That lemma yields n with $n > x \geq C$, $n > 0$ and $g(n) > n^{1-\varepsilon}$. □

Lemma 3.21 (`infinite_of_lt_one`)**.** *Assume (H) and $0 < \varepsilon < 1$. Then $\{n \in \mathbb{N} : 0 < n,\ g(n) > n^{1-\varepsilon}\}$ is infinite.*

Lean: `infinite_of_lt_one` — proved by AIProver, job 2, session 0

```
theorem infinite_of_lt_one (hH : SmoothShiftedPrimesHypothesis) (ε : ℝ)
    (hε : 0 < ε) (hε1 : ε < 1) :
    Set.Infinite {n : ℕ | 0 < n ∧ ((totientFiberCard n : ℕ) : ℝ) > (n : ℝ) ^ (1 - ε)} := by



  by_contra! h_fin

  have h_fin_set : Set.Finite {n : ℕ | 0 < n ∧ ((totientFiberCard n : ℕ) : ℝ) > (n : ℝ) ^ (1 - ε)} := by
    rwa [Set.not_infinite] at h_fin
  obtain ⟨M, hM⟩ := Set.Finite.bddAbove h_fin_set


  obtain ⟨n, hnC, hnpos, hn⟩ := exists_good_n_large hH ε hε hε1 (max (M : ℝ) 0)
  have hn_gt_M : (M : ℝ) < (n : ℝ) := by
    have : max (M : ℝ) 0 < (n : ℝ) := hnC
    have hM_nonneg : 0 ≤ (M : ℝ) := Nat.cast_nonneg _
    have : (M : ℝ) ≤ max (M : ℝ) (0 : ℝ) := le_max_left _ _
    linarith
  have hn_nat_gt_M : M < n := by exact_mod_cast hn_gt_M
  have hM_bound : n ≤ M := hM (Set.mem_setOf.mpr ⟨hnpos, hn⟩)
  exact hn_nat_gt_M.not_le hM_bound
```

Proof. If the set were finite, it would be bounded above by some $M \in \mathbb{N}$. Lemma 3.20 with $C = \max(M, 0)$ produces an element $n > M$ of the set, a contradiction. □

3.2 Proof of the theorem

Lean: `solution` — main proof from the captain's sketch

```
theorem solution
    (hH : SmoothShiftedPrimesHypothesis) (ε : ℝ) (hε : 0 < ε) :
    Set.Infinite {n : ℕ | 0 < n ∧ ((totientFiberCard n : ℕ) : ℝ) > (n : ℝ) ^ (1 - ε)} := by
  by_cases h1 : ε < 1
  · exact infinite_of_lt_one hH ε hε h1
  · push_neg at h1
    have h := infinite_of_lt_one hH (1 / 2) (by norm_num) (by norm_num)
    apply h.mono
    intro n hn
    simp only [Set.mem_setOf_eq] at hn ⊢
    refine ⟨hn.1, lt_of_le_of_lt ?_ hn.2⟩
    apply Real.rpow_le_rpow_of_exponent_le
    · exact Nat.one_le_cast.mpr hn.1
    · linarith
```

Proof. Assume (H) and let $\varepsilon > 0$.

If $\varepsilon < 1$, the claim is Lemma 3.21.

If $\varepsilon \geq 1$, apply Lemma 3.21 with $\varepsilon' = 1/2$. For $n \geq 1$ the map $t \mapsto n^t$ is nondecreasing, and $1 - \varepsilon \leq 0 < 1/2$. Hence every $n > 0$ with $g(n) > n^{1/2}$ also satisfies

$$g(n) > n^{1/2} \geq n^{1-\varepsilon}.$$

The infinite set for $\varepsilon' = 1/2$ is therefore contained in the set for ε, which is consequently infinite. □

4 Independent review

A referee model (claude-opus-5-5), which took no part in the formalization or the proof, compared the verified Lean file with the source. Verdict: **FAITHFUL**.

Definitions. `totientFiberCard` encodes $g(n) = \#\{m \geq 1 : \varphi(m) = n\}$ as the `Nat.card` of the subtype $\{m : 0 < m \wedge \varphi(m) = n\}$. `Nat.card` returns 0 on an infinite type. That case does not arise, because the file proves the fiber finite (`totient_fiber_finite`). At $n = 0$ the fiber is empty, since $\varphi(m) = 0$ only for $m = 0$. This agrees with the source, which only considers positive n.

`IsSmoothNat` m y says that every prime factor of m is at most y. For $m > 1$ this is exactly $P^+(m) \leq y$. For $m = 1$ it holds vacuously, which matches the convention $P^+(1) = 1$. In the hypothesis this matters only for $p = 2$, which affects nothing.

`SmoothShiftedPrimesHypothesis` is Hypothesis (H) verbatim, with real x and real powers. It says: for all real $\delta \in (0, 1/4)$ and $\eta > 0$ there is x_0 such that for all $x \geq x_0$,

$$\#\{p \text{ prime} : 2x < p \leq 5x,\ p - 1 \text{ is } x^{\delta}\text{-smooth}\} \ \geq\ x^{1-\eta}.$$

The counted set is bounded by $5x$, so `Nat.card` is the true cardinality. The hypothesis is not trivially false: it is a genuine smooth-shifted-primes statement. It is therefore not a vacuous assumption.

Statement. The theorem assumes `SmoothShiftedPrimesHypothesis` and any real $\varepsilon > 0$. It concludes that $\{n \in \mathbb{N} : 0 < n \wedge g(n) > n^{1-\varepsilon}\}$ is infinite. This is exactly the source theorem: it is conditional on (H), the quantification is over all $\varepsilon > 0$, and the inequality is strict.

There are no extra hypotheses. Nothing is weakened. Nothing needs to be supplied to recover the source statement.

The parenthetical Lemma 1 is not part of the target statement. Only its finiteness half appears, as the auxiliary lemma `totient_fiber_finite`. The bound $g(n) \ll_\eta n^{1+\eta}$ is not formalized. It is not needed for the main conclusion.

Proof. The formal proof follows the source's product-and-pigeonhole argument, with different scaling and constants.

`exists_prime_finset` applies (H) with $\eta/2$. It extracts a finset S of primes $p \in (2x, 5x]$ with $p - 1$ being x^{δ}-smooth and $|S| \geq x^{1-\eta}$. The proof takes $\delta = \eta = \varepsilon/8$ and $k = \lfloor x^{2\delta} \rfloor$.

`choose_ge_pow_div` proves $\binom{M}{k} \geq (M/k)^k$. This is inequality (1).

`smooth_count` bounds the number of y-smooth integers in $[1, N]$ by $(\lfloor \log_2 N \rfloor + 1)^{\lfloor y \rfloor}$. This is inequality (2), in a slightly sharper form.

`totient_prod_primes` gives $\varphi(\prod_{p \in Q} p) = \prod_{p \in Q}(p - 1)$. `prod_primes_inj` is unique factorization. `prod_smooth_bounded` shows that the product of the shifted primes is smooth and at most N^k, where $N = \lfloor 5x \rfloor$.

`exists_large_fiber` is the pigeonhole step:

$$\binom{|S|}{k} \leq |V| \cdot g(n)$$

for some value n.

The asymptotic lemmas (`asymp_log_ineq_growth`, `asymp_log_ineq_pointwise`, `asymp_main`) make the source's $o(k)$ estimate explicit by comparing logarithms. They show

$$|V| \cdot (N^k)^{1-\varepsilon} < \binom{M}{k}$$

for large x, using $4\delta < \varepsilon$. `good_n_from_finset` combines these to obtain n with $g(n) > n^{1-\varepsilon}$.

One deviation from the source: infinitude is derived from $n \geq p_0 - 1 > x$, where p_0 is a prime in the chosen subset. The source instead appeals to the upper bound in Lemma 1. The formal route is simpler and valid.

`infinite_of_lt_one` turns arbitrarily large solutions into an infinite set. The theorem `solution` handles $\varepsilon \geq 1$ by monotonicity from $\varepsilon = 1/2$, using $n^{1-\varepsilon} \leq n^{1/2}$ for $n \geq 1$, as in the source.

All auxiliary lemmas are stated at least as strongly as the steps they represent.

Concerns.

- The quantitative bound $g(n) \ll_\eta n^{1+\eta}$ from Lemma 1 is not formalized. Only the finiteness of fibers is proved. The main theorem does not depend on the bound.
- The result is conditional on (H), exactly as in the source. The formalization does not establish (H).

A Complete Lean file

The verified file, as checked by Lean.

```
import Mathlib
set_option autoImplicit false

/-- `g(n) = #{m ≥ 1 : φ(m) = n}`. The fiber is finite (Lemma 1). If it were infinite,
`Nat.card` would return `0`, which could only make the theorem harder. -/
noncomputable def totientFiberCard (n : ℕ) : ℕ :=
  Nat.card {m : ℕ // 0 < m ∧ Nat.totient m = n}

/-- `m` is `y`-smooth: every prime factor of `m` is at most `y`.
(For `m > 1` this is `P⁺(m) ≤ y`.) -/
def IsSmoothNat (m : ℕ) (y : ℝ) : Prop :=
  ∀ q ∈ m.primeFactors, (q : ℝ) ≤ y

/-- Hypothesis (H): for every `0 < δ < 1/4` and every `η > 0` there is `x₀` such that
for all real `x ≥ x₀`, `#{p prime : 2x < p ≤ 5x, P⁺(p-1) ≤ x^δ} ≥ x^(1-η)`. -/
def SmoothShiftedPrimesHypothesis : Prop :=
  ∀ δ : ℝ, 0 < δ → δ < 1 / 4 → ∀ η : ℝ, 0 < η →
    ∃ x₀ : ℝ, ∀ x : ℝ, x₀ ≤ x →
      (Nat.card {p : ℕ // p.Prime ∧ 2 * x < (p : ℝ) ∧ (p : ℝ) ≤ 5 * x ∧
          IsSmoothNat (p - 1) (x ^ δ)} : ℝ) ≥ x ^ (1 - η)



theorem totient_fiber_finite (n : ℕ) :
    {m : ℕ | 0 < m ∧ Nat.totient m = n}.Finite := by
  by_cases hn : n = 0
  · subst hn
    have h_empty : {m : ℕ | 0 < m ∧ Nat.totient m = 0} = (∅ : Set ℕ) := by
      ext m; simp; intro hm; omega
    rw [h_empty]
    exact Set.finite_empty
  have hnpos : 0 < n := Nat.pos_of_ne_zero hn

  let B : ℕ := (n + 1) ^ ((n + 2) * n)
  have hB_set : Set.Finite {m : ℕ | m ≤ B} := by
```

```
    refine Set.Finite.subset (Finset.finite_toSet (Finset.range (B + 1))) ?_
    intro m hm
    have hm_le_B : m ≤ B := hm
    have hm_lt : m < B + 1 := Nat.lt_succ_of_le hm_le_B
    exact Finset.mem_range.mpr hm_lt
  have h_bound : ∀ m, 0 < m → Nat.totient m = n → m ≤ B := by
    intro m hmpos hm_tot
    have hm_ne_zero : m ≠ 0 := by omega

    have hm_factor : m = ∏ p ∈ m.primeFactors, p ^ (m.factorization p) := by
      calc
        m = m.factorization.prod (· ^ ·) := by
          rw [Nat.factorization_prod_pow_eq_self hm_ne_zero]
        _ = ∏ p ∈ m.primeFactors, p ^ (m.factorization p) := by
          rw [Nat.prod_factorization_eq_prod_primeFactors]

    have two_pow_ge_succ : ∀ k : ℕ, k + 1 ≤ 2 ^ k := by
      intro k
      induction' k with k ih
      · simp
      · rw [pow_succ]
        have h : k + 2 ≤ 2 * (k + 1) := by omega
        calc
          k + 2 ≤ 2 * (k + 1) := h
          _ ≤ 2 * (2 ^ k) := Nat.mul_le_mul_left 2 ih
          _ = 2 ^ (k + 1) := by ring

    have h_prime_dvd : ∀ p ∈ m.primeFactors, p - 1 ∣ n := by
      intro p hp
      have hp_prime : Nat.Prime p := Nat.prime_of_mem_primeFactors hp
      let a := m.factorization p
      have ha_pos : 0 < a := by
        have h_ne_zero : m.factorization p ≠ 0 := by
          have : p ∈ (m.factorization).support := by
            rw [Nat.support_factorization]; exact hp
          rw [Finsupp.mem_support_iff] at this
          exact this
        omega
      have h_tot_pow_dvd : Nat.totient (p ^ a) ∣ Nat.totient m := by
        apply Nat.totient_dvd_of_dvd
        rw [hm_factor]
        exact Finset.dvd_prod_of_mem (λ x => x ^ (m.factorization x)) hp
      rw [hm_tot] at h_tot_pow_dvd
      have h_tot_pow_eq : Nat.totient (p ^ a) = p ^ (a - 1) * (p - 1) :=
        Nat.totient_prime_pow hp_prime ha_pos
      rw [h_tot_pow_eq] at h_tot_pow_dvd
      exact dvd_trans (dvd_mul_left _ _) h_tot_pow_dvd
    have h_prime_le : ∀ p ∈ m.primeFactors, p ≤ n + 1 := by
      intro p hp
      have hp_sub_one_dvd : p - 1 ∣ n := h_prime_dvd p hp
      have h_le : p - 1 ≤ n := Nat.le_of_dvd hnpos hp_sub_one_dvd
      omega

    have h_exponent_le : ∀ p ∈ m.primeFactors, m.factorization p ≤ n := by
      intro p hp
      have hp_prime : Nat.Prime p := Nat.prime_of_mem_primeFactors hp
      have hp_ge_two : 2 ≤ p := Nat.Prime.two_le hp_prime
      let a := m.factorization p
      have ha_pos : 0 < a := by
```

```
      have h_ne_zero : m.factorization p ≠ 0 := by
        have : p ∈ (m.factorization).support := by
          rw [Nat.support_factorization]; exact hp
        rw [Finsupp.mem_support_iff] at this
        exact this
      omega
    have h_tot_pow_dvd : Nat.totient (p ^ a) ∣ Nat.totient m := by
      apply Nat.totient_dvd_of_dvd
      rw [hm_factor]
      exact Finset.dvd_prod_of_mem (λ x => x ^ (m.factorization x)) hp
    rw [hm_tot] at h_tot_pow_dvd
    have h_tot_pow_eq : Nat.totient (p ^ a) = p ^ (a - 1) * (p - 1) :=
      Nat.totient_prime_pow hp_prime ha_pos
    rw [h_tot_pow_eq] at h_tot_pow_dvd
    have h_dvd_pow : p ^ (a - 1) ∣ n := by
      have h : p ^ (a - 1) ∣ p ^ (a - 1) * (p - 1) := dvd_mul_right _ _
      exact dvd_trans h h_tot_pow_dvd

    have ha_le_pow : a ≤ p ^ (a - 1) := by
      have h_two_ge : a ≤ 2 ^ (a - 1) := by
        have h := two_pow_ge_succ (a - 1)
        omega
      have h_pow_ge : 2 ^ (a - 1) ≤ p ^ (a - 1) := Nat.pow_le_pow_left hp_ge_two (a - 1)
      exact Nat.le_trans h_two_ge h_pow_ge
    have h_pow_le_n : p ^ (a - 1) ≤ n := Nat.le_of_dvd hnpos h_dvd_pow
    omega

  rw [hm_factor]

  have h_pow_le : ∀ p ∈ m.primeFactors, p ^ (m.factorization p) ≤ (n + 1) ^ (m.factorization p) := by
    intro p hp
    exact Nat.pow_le_pow_left (h_prime_le p hp) _
  have h_prod_le : (∏ p ∈ m.primeFactors, p ^ (m.factorization p)) ≤
                   (∏ p ∈ m.primeFactors, (n + 1) ^ (m.factorization p)) :=
    Finset.prod_le_prod' h_pow_le

  have h_prod_pow : (∏ p ∈ m.primeFactors, (n + 1) ^ (m.factorization p)) =
                    (n + 1) ^ (∑ p ∈ m.primeFactors, m.factorization p) := by
    rw [Finset.prod_pow_eq_pow_sum]

  have h_card_le : m.primeFactors.card ≤ n + 2 := by
    have h_subset : m.primeFactors ⊆ Finset.range (n + 2) := by
      intro x hx
      apply Finset.mem_range.mpr
      have hx_le : x ≤ n + 1 := h_prime_le x hx
      omega
    have h_card_range : (Finset.range (n + 2)).card = n + 2 := Finset.card_range (n + 2)
    have h_card_le_range : m.primeFactors.card ≤ (Finset.range (n + 2)).card :=
      Finset.card_le_card h_subset
    calc
      m.primeFactors.card ≤ (Finset.range (n + 2)).card := h_card_le_range
      _ = n + 2 := h_card_range
  have h_sum_le : (∑ p ∈ m.primeFactors, m.factorization p) ≤ (n + 2) * n := by
    calc
      (∑ p ∈ m.primeFactors, m.factorization p) ≤ (∑ p ∈ m.primeFactors, n) :=
        Finset.sum_le_sum (λ p hp => h_exponent_le p hp)
      _ = (m.primeFactors.card : ℕ) * n := by
        rw [Finset.sum_const_nat]
        intro x hx; rfl
```

```
        _ ≤ (n + 2) * n := Nat.mul_le_mul_right n h_card_le

    have h_pow_le' : (n + 1) ^ (∑ p ∈ m.primeFactors, m.factorization p) ≤ (n + 1) ^ ((n + 2) * n) :=
      Nat.pow_le_pow_right (by omega) h_sum_le
    calc
      (∏ p ∈ m.primeFactors, p ^ (m.factorization p)) ≤
          (∏ p ∈ m.primeFactors, (n + 1) ^ (m.factorization p)) := h_prod_le
      _ = (n + 1) ^ (∑ p ∈ m.primeFactors, m.factorization p) := h_prod_pow
      _ ≤ (n + 1) ^ ((n + 2) * n) := h_pow_le'

  have h_subset : {m : ℕ | 0 < m ∧ Nat.totient m = n} ⊆ {m : ℕ | m ≤ B} := by
    intro m hm
    simp [h_bound m hm.1 hm.2]
  exact hB_set.subset h_subset

theorem card_le_totientFiberCard (n : ℕ) (T : Finset ℕ)
    (hT : ∀ m ∈ T, 0 < m ∧ Nat.totient m = n) :
    T.card ≤ totientFiberCard n := by
  have h_fin : {m : ℕ | 0 < m ∧ Nat.totient m = n}.Finite := totient_fiber_finite n
  haveI : Fintype {m : ℕ // 0 < m ∧ Nat.totient m = n} := h_fin.fintype
  have h_card_eq : totientFiberCard n = Fintype.card {m : ℕ // 0 < m ∧ Nat.totient m = n} := by
    simp [totientFiberCard, Nat.card_eq_fintype_card]
  rw [h_card_eq]
  let f : T → {m : ℕ // 0 < m ∧ Nat.totient m = n} := λ m => ⟨m.1, hT m.1 m.2⟩
  have hf_inj : Function.Injective f := by
    intro a b h
    have hval : (f a).1 = (f b).1 := by rw [h]
    apply Subtype.ext
    simpa using hval
  let S : Finset T := Finset.univ
  have h_card_T : S.card = T.card := by
    calc
      S.card = Fintype.card {x // x ∈ T} := by simp [S]
      _ = T.card := by simp
  have h_image_card : (Finset.image f S).card = S.card :=
    Finset.card_image_of_injective S hf_inj
  have h_sub : Finset.image f S ⊆ (Finset.univ : Finset {m : ℕ // 0 < m ∧ Nat.totient m = n}) :=
    Finset.subset_univ _
  have h_card_le : (Finset.image f S).card ≤ (Finset.univ : Finset {m : ℕ // 0 < m ∧ Nat.totient m =
↪  n}).card :=
    Finset.card_le_card h_sub
  have h_univ_card : (Finset.univ : Finset {m : ℕ // 0 < m ∧ Nat.totient m = n}).card = Fintype.card {m : ℕ
↪  // 0 < m ∧ Nat.totient m = n} := by
    simp
  calc
    T.card = S.card := by symm; exact h_card_T
    _ = (Finset.image f S).card := by symm; exact h_image_card
    _ ≤ (Finset.univ : Finset {m : ℕ // 0 < m ∧ Nat.totient m = n}).card := h_card_le
    _ = Fintype.card {m : ℕ // 0 < m ∧ Nat.totient m = n} := h_univ_card

theorem totient_prod_primes (Q : Finset ℕ) (hQ : ∀ p ∈ Q, p.Prime) :
    Nat.totient (∏ p ∈ Q, p) = ∏ p ∈ Q, (p - 1) := by
  induction' Q using Finset.induction_on with p Q hp ih
  · simp
  · have hp_prime : p.Prime := hQ p (Finset.mem_insert_self p Q)
    have hQ' : ∀ q ∈ Q, q.Prime := fun q hq => hQ q (Finset.mem_insert_of_mem hq)
    rw [Finset.prod_insert hp, Finset.prod_insert hp]
    have hcop : (∏ x ∈ Q, x).Coprime p := by
      refine Nat.Coprime.prod_left fun q hq => ?_
```

```
      have hq_prime : q.Prime := hQ' q hq
      refine hq_prime.coprime_iff_not_dvd.mpr fun h => ?_
      have hq_one_lt : 1 < q := hq_prime.one_lt
      have hp_eq_q : p = q := ((hp_prime.dvd_iff_eq hq_one_lt.ne').mp h)
      exact hp (hp_eq_q ⁸ hq)
    rw [← mul_comm, Nat.totient_mul hcop, mul_comm, ih hQ', Nat.totient_prime hp_prime, mul_comm]

theorem prod_primes_inj (Q₁ Q₂ : Finset ℕ) (h₁ : ∀ p ∈ Q₁, p.Prime)
    (h₂ : ∀ p ∈ Q₂, p.Prime) (h : ∏ p ∈ Q₁, p = ∏ p ∈ Q₂, p) : Q₁ = Q₂ := by
  calc
    Q₁ = Nat.primeFactors (∏ p ∈ Q₁, p) := by
      rw [Nat.primeFactors_prod h₁]
    _ = Nat.primeFactors (∏ p ∈ Q₂, p) := by rw [h]
    _ = Q₂ := by rw [Nat.primeFactors_prod h₂]

theorem exists_large_fiber (S : Finset ℕ) (hS : ∀ p ∈ S, p.Prime) (k : ℕ)
    (hk : k ≤ S.card) (V : Finset ℕ)
    (hV : ∀ Q ⊆ S, Q.card = k → ∏ p ∈ Q, (p - 1) ∈ V) :
    ∃ Q ⊆ S, Q.card = k ∧
      S.card.choose k ≤ V.card * totientFiberCard (∏ p ∈ Q, (p - 1)) := by
  let T := Finset.powersetCard k S
  have hTcard : T.card = S.card.choose k := by simp [T]
  let f : Finset ℕ → ℕ := fun Q => ∏ p ∈ Q, (p - 1)
  have hf_maps_to : ∀ Q ∈ T, f Q ∈ V := by
    intro Q hQ
    rcases Finset.mem_powersetCard.mp hQ with ⟨hQsub, hQcard⟩
    exact hV Q hQsub hQcard
  have h_sum_eq : T.card = ∑ n ∈ V, (Finset.filter (fun Q => f Q = n) T).card := by
    have h_mapsTo : Set.MapsTo f (T : Set (Finset ℕ)) (V : Set ℕ) := by
      intro Q hQ
      simp [hf_maps_to Q hQ]
    simpa using Finset.card_eq_sum_card_fiberwise h_mapsTo
  by_cases hVempty : V = ∅
  · rw [hVempty] at hf_maps_to
    have hTempty : T = ∅ := by
      by_contra hTnonempty
      obtain ⟨Q, hQ⟩ := Finset.nonempty_iff_ne_empty.mpr hTnonempty
      have h_in_V := hf_maps_to Q hQ
      simp at h_in_V
    rw [hTempty] at hTcard
    simp at hTcard
    have hpos : 0 < S.card.choose k := Nat.choose_pos hk
    rw [hTcard] at hpos
    exact absurd hpos (lt_irrefl _)
  · have hVnonempty : V.Nonempty :=
      Finset.nonempty_iff_ne_empty.mpr hVempty
    have h_exists_max : ∃ n ∈ V, ∀ n' ∈ V,
      (Finset.filter (fun Q => f Q = n') T).card ≤ (Finset.filter (fun Q => f Q = n) T).card := by
      simpa using Finset.exists_max_image V (fun n => (Finset.filter (fun Q => f Q = n) T).card) hVnonempty
    rcases h_exists_max with ⟨n, hnV, hn_max⟩
    have h_sum_le : ∑ n' ∈ V, (Finset.filter (fun Q => f Q = n') T).card ≤
      V.card * (Finset.filter (fun Q => f Q = n) T).card := by
      calc
        ∑ n' ∈ V, (Finset.filter (fun Q => f Q = n') T).card
            ≤ ∑ n' ∈ V, (Finset.filter (fun Q => f Q = n) T).card :=
          Finset.sum_le_sum (fun n' hn' => hn_max n' hn')
        _ = V.card * (Finset.filter (fun Q => f Q = n) T).card := by simp
    have h_ineq : T.card ≤ V.card * (Finset.filter (fun Q => f Q = n) T).card := by
      linarith
```

```
    let fiber := Finset.filter (fun Q => f Q = n) T
    by_cases h_fiber_card_zero : fiber.card = 0
    · rw [h_fiber_card_zero, mul_zero] at h_ineq
      have hTpos : 0 < T.card := by
        rw [hTcard]
        exact Nat.choose_pos hk
      have : ¬ T.card ≤ 0 := Nat.not_le_of_lt hTpos
      exact absurd h_ineq this
    · have h_fiber_card_pos : 0 < fiber.card :=
        Nat.pos_of_ne_zero h_fiber_card_zero
      have h_fiber_nonempty : fiber.Nonempty := Finset.card_pos.mp h_fiber_card_pos
      obtain ⟨Q, hQ⟩ := h_fiber_nonempty
      refine ⟨Q, ?_, ?_, ?_⟩
      · rcases Finset.mem_filter.mp hQ with ⟨hQT, hQf⟩
        rcases Finset.mem_powersetCard.mp hQT with ⟨hQsub, hQcard⟩
        exact hQsub
      · rcases Finset.mem_filter.mp hQ with ⟨hQT, hQf⟩
        rcases Finset.mem_powersetCard.mp hQT with ⟨hQsub, hQcard⟩
        exact hQcard
      · have hfQ : f Q = n := by
          rcases Finset.mem_filter.mp hQ with ⟨_, hQf⟩
          exact hQf
        rw [← hTcard, show totientFiberCard (∏ p ∈ Q, (p - 1)) = totientFiberCard (f Q) by
          simp [f], hfQ]
        apply le_trans h_ineq
        apply Nat.mul_le_mul_left V.card
        have h_card_le : fiber.card ≤ totientFiberCard n := by
          have hmem : ∀ Q ∈ fiber, Q ⊆ S ∧ Q.card = k ∧ ∏ p ∈ Q, (p - 1) = n := by
            intro Q hQ
            rcases Finset.mem_filter.mp hQ with ⟨hQT, hQf⟩
            rcases Finset.mem_powersetCard.mp hQT with ⟨hQsub, hQcard⟩
            exact ⟨hQsub, hQcard, hQf⟩
          have hinj : Set.InjOn (fun Q : Finset ℕ => ∏ p ∈ Q, p) (fiber : Set (Finset ℕ)) := by
            intro Q1 hQ1 Q2 hQ2 h
            have h1 := hmem Q1 (Finset.mem_coe.mp hQ1)
            have h2 := hmem Q2 (Finset.mem_coe.mp hQ2)
            exact prod_primes_inj Q1 Q2 (fun p hp => hS p (h1.1 hp))
              (fun p hp => hS p (h2.1 hp)) h
          calc fiber.card = (fiber.image (fun Q : Finset ℕ => ∏ p ∈ Q, p)).card :=
                (Finset.card_image_of_injOn hinj).symm
            _ ≤ totientFiberCard n := by
              apply card_le_totientFiberCard
              intro m hm
              obtain ⟨Q', hQ', rfl⟩ := Finset.mem_image.mp hm
              have hQ'' := hmem Q' hQ'
              show 0 < ∏ p ∈ Q', p ∧ Nat.totient (∏ p ∈ Q', p) = n
              refine ⟨Finset.prod_pos (fun p hp => (hS p (hQ''.1 hp)).pos), ?_⟩
              rw [totient_prod_primes Q' (fun p hp => hS p (hQ''.1 hp))]
              exact hQ''.2.2
        exact h_card_le

theorem smooth_count (N : ℕ) (y : ℝ) :
    ∃ V : Finset ℕ, V.card ≤ (Nat.log 2 N + 1) ^ ⌊y⌋₊ ∧
      ∀ n : ℕ, 0 < n → n ≤ N → IsSmoothNat n y → n ∈ V := by
  classical
  obtain ⟨P, hP⟩ : ∃ P : Finset ℕ, P = (Finset.Icc 1 ⌊y⌋₊).filter Nat.Prime := ⟨_, rfl⟩
  have hPprime : ∀ q ∈ P, q.Prime := by
    intro q hq
    rw [hP, Finset.mem_filter] at hq
```

```
    exact hq.2
  refine ⟨(Fintype.piFinset (fun _ : P => Finset.range (Nat.log 2 N + 1))).image
      (fun f => ∏ p : P, (p : ℕ) ^ (f p)), ?_, ?_⟩
  · calc _ ≤ (Fintype.piFinset (fun _ : P => Finset.range (Nat.log 2 N + 1))).card :=
          Finset.card_image_le
      _ = (Nat.log 2 N + 1) ^ P.card := by
          rw [Fintype.card_piFinset, Finset.prod_const, Finset.card_univ, Fintype.card_coe,
            Finset.card_range]
      _ ≤ (Nat.log 2 N + 1) ^ y := by
          apply Nat.pow_le_pow_right (by omega)
          calc P.card ≤ (Finset.Icc 1 y).card := by
                rw [hP]; exact Finset.card_filter_le _ _
            _ = y := by simp
  · intro n hn hnN hsm
    rw [Finset.mem_image]
    refine ⟨fun p => n.factorization p, ?_, ?_⟩
    · rw [Fintype.mem_piFinset]
      intro p
      rw [Finset.mem_range, Nat.lt_add_one_iff]
      apply Nat.le_log_of_pow_le (by norm_num)
      have hp : (p : ℕ).Prime := hPprime p.1 p.2
      calc 2 ^ n.factorization p ≤ (p : ℕ) ^ n.factorization p :=
            Nat.pow_le_pow_left hp.two_le _
        _ ≤ n := Nat.le_of_dvd hn
            ((Nat.Prime.pow_dvd_iff_le_factorization hp hn.ne').mpr le_rfl)
        _ ≤ N := hnN
    · show ∏ p : P, (p : ℕ) ^ n.factorization p = n
      have hc := Finset.prod_coe_sort P (fun p => p ^ n.factorization p)
      simp only at hc
      rw [hc]
      conv_rhs => rw [← Nat.factorization_prod_pow_eq_self hn.ne']
      rw [Finsupp.prod, Nat.support_factorization]
      symm
      apply Finset.prod_subset
      · intro q hq
        rw [hP, Finset.mem_filter, Finset.mem_Icc]
        have hq' := Nat.prime_of_mem_primeFactors hq
        exact ⟨⟨hq'.one_lt.le, Nat.le_floor (hsm q hq)⟩, hq'⟩
      · intro q _ hq
        show q ^ n.factorization q = 1
        have h0 : n.factorization q = 0 := by
          by_contra h
          exact hq (by rw [← Nat.support_factorization, Finsupp.mem_support_iff]; exact h)
        rw [h0, pow_zero]

theorem prod_smooth_bounded (Q : Finset ℕ) (y : ℝ) (B : ℕ)
    (h : ∀ p ∈ Q, 0 < p - 1 ∧ p - 1 ≤ B ∧ IsSmoothNat (p - 1) y) :
    0 < ∏ p ∈ Q, (p - 1) ∧ ∏ p ∈ Q, (p - 1) ≤ B ^ Q.card ∧
      IsSmoothNat (∏ p ∈ Q, (p - 1)) y := by
  have hpos : ∀ p ∈ Q, 0 < p - 1 := λ p hp => (h p hp).1
  have hle : ∀ p ∈ Q, p - 1 ≤ B := λ p hp => (h p hp).2.1
  have hsm : ∀ p ∈ Q, IsSmoothNat (p - 1) y := λ p hp => (h p hp).2.2
  have hpos_prod : 0 < ∏ p ∈ Q, (p - 1) := by
    refine Finset.prod_pos ?_
    intro p hp
    exact hpos p hp
  have h_le_prod : ∏ p ∈ Q, (p - 1) ≤ B ^ Q.card := by
    calc
      ∏ p ∈ Q, (p - 1) ≤ ∏ p ∈ Q, B := Finset.prod_le_prod' (by
```

```
        intro p hp
        have hp_pos := hpos p hp
        have hble := hle p hp
        have : (p - 1 : ℕ) ≤ B := hble
        exact this
      )
      _ = B ^ Q.card := by simp
  have hnz : ∏ p ∈ Q, (p - 1) ≠ 0 := by
    have : 0 < ∏ p ∈ Q, (p - 1) := hpos_prod
    omega
  have hsm_prod : IsSmoothNat (∏ p ∈ Q, (p - 1)) y := by
    intro q hq
    have hq_mem_ps : q ∈ (∏ p ∈ Q, (p - 1)).primeFactors := hq
    have hq_prime : q.Prime := Nat.prime_of_mem_primeFactors hq_mem_ps
    have hq_dvd : q ∣ ∏ p ∈ Q, (p - 1) := Nat.dvd_of_mem_primeFactors hq_mem_ps

    rcases (Prime.dvd_finset_prod_iff (hq_prime.prime) (λ p => p - 1)).mp hq_dvd with ⟨p, hp, hq_dvd_p⟩
    have hsm_p := hsm p hp

    have hq_mem_factor : q ∈ (p - 1).primeFactors := by
      have hp_pos := hpos p hp
      have hp1_ne_zero : p - 1 ≠ 0 := by omega
      rw [Nat.mem_primeFactors_of_ne_zero hp1_ne_zero]
      exact ⟨hq_prime, hq_dvd_p⟩

    exact hsm_p q hq_mem_factor
  exact And.intro hpos_prod (And.intro h_le_prod hsm_prod)

open Finset Nat in
theorem choose_ge_pow_div_aiprover_s9 (M k : ℕ) (hk : k ≤ M) :
    ((M : ℝ) / k) ^ k ≤ (M.choose k : ℝ) := by
  by_cases hkz : k = 0
  · subst hkz; simp
  · have hkpos_nat : 0 < k := Nat.pos_of_ne_zero hkz
    have hkpos' : (k : ℝ) > 0 := by exact mod_cast hkpos_nat



    have h_factor (j : ℕ) (hj : j < k) : (M : ℝ) - (j : ℝ) ≥ ((M : ℝ) / k) * ((k : ℝ) - (j : ℝ)) := by
      have hkM : (k : ℝ) ≤ (M : ℝ) := by exact mod_cast hk
      have hMk : (M : ℝ) / k ≥ 1 := ((one_le_div hkpos').mpr hkM)
      calc
        (M : ℝ) - (j : ℝ) = (M : ℝ) - (j : ℝ) := rfl
        _ ≥ (M : ℝ) - (M : ℝ) / k * (j : ℝ) := by
          have : (M : ℝ) / k * (j : ℝ) ≥ 1 * (j : ℝ) := by nlinarith
          nlinarith
        _ = (M : ℝ) / k * k - (M : ℝ) / k * (j : ℝ) := by
          field_simp [hkpos'.ne']
        _ = ((M : ℝ) / k) * ((k : ℝ) - (j : ℝ)) := by ring


    have h_desc_fact_ℕ : M.descFactorial k = k ! * M.choose k := by
      rw [descFactorial_eq_factorial_mul_choose]

    have h_prod_desc_ℕ : M.descFactorial k = ∏ i ∈ range k, (M - i) := by
      rw [descFactorial_eq_prod_range]


    have h_prod_desc_ℝ : (M.descFactorial k : ℝ) = ∏ i ∈ range k, ((M : ℝ) - (i : ℝ)) := by
```

```
  have h := congrArg (Nat.cast : ℕ → ℝ) h_prod_desc_ℕ

  rw [h, Nat.cast_prod]

  refine Finset.prod_congr rfl (fun i hi => ?_)
  have hi_lt_k : i < k := Finset.mem_range.1 hi
  have hiM : i ≤ M := Nat.le_trans (Nat.le_of_lt hi_lt_k) hk
  rw [Nat.cast_sub hiM]

have h_desc_fact_ℝ : (M.descFactorial k : ℝ) = (k ! : ℝ) * (M.choose k : ℝ) := by
  exact mod_cast h_desc_fact_ℕ


have h_choose_eq : (M.choose k : ℝ) = (∏ i ∈ range k, ((M : ℝ) - (i : ℝ))) / (k ! : ℝ) := by
  have hk_fact_ne_zero : (k ! : ℝ) ≠ 0 := by exact mod_cast factorial_ne_zero k
  field_simp [hk_fact_ne_zero]
  nlinarith


have h_prod_factor_ge : (∏ i ∈ range k, ((M : ℝ) - (i : ℝ))) ≥
    (∏ i ∈ range k, (((M : ℝ) / k) * ((k : ℝ) - (i : ℝ)))) := by
  apply Finset.prod_le_prod
  · intro i hi
    have hi_lt_k : i < k := Finset.mem_range.1 hi
    have h_nonneg_k_sub_i : 0 ≤ (k : ℝ) - (i : ℝ) := by
      have : (i : ℝ) ≤ (k : ℝ) := by exact mod_cast hi_lt_k.le
      nlinarith
    have hM_nonneg : 0 ≤ (M : ℝ) := by exact mod_cast Nat.zero_le M
    have h_div_nonneg : 0 ≤ (M : ℝ) / k := div_nonneg hM_nonneg (by positivity)
    exact mul_nonneg h_div_nonneg h_nonneg_k_sub_i
  · intro i hi
    have hi_lt_k : i < k := Finset.mem_range.1 hi
    exact h_factor i hi_lt_k


have h_prod_split : (∏ i ∈ range k, (((M : ℝ) / k) * ((k : ℝ) - (i : ℝ)))) =
    ((M : ℝ) / k) ^ k * (k ! : ℝ) := by
  calc
    (∏ i ∈ range k, (((M : ℝ) / k) * ((k : ℝ) - (i : ℝ)))) =
        (∏ i ∈ range k, ((M : ℝ) / k)) * (∏ i ∈ range k, ((k : ℝ) - (i : ℝ))) := by
      rw [Finset.prod_mul_distrib]
    _ = (((M : ℝ) / k) ^ k) * (∏ i ∈ range k, ((k : ℝ) - (i : ℝ))) := by

      simp [div_pow]
    _ = (((M : ℝ) / k) ^ k) * ((k.descFactorial k : ℝ)) := by
      rw [descFactorial_eq_prod_range, Nat.cast_prod]
      refine congrArg (fun p : ℝ => ((M : ℝ) / k) ^ k * p) (Finset.prod_congr rfl (fun i hi => ?_))
      have hi_lt_k : i < k := Finset.mem_range.1 hi
      rw [Nat.cast_sub (Nat.le_of_lt hi_lt_k)]
    _ = ((M : ℝ) / k) ^ k * (k ! : ℝ) := by rw [descFactorial_self]


have h_main : ((M : ℝ) / k) ^ k * (k ! : ℝ) ≤ (∏ i ∈ range k, ((M : ℝ) - (i : ℝ))) := by
  nlinarith


rw [h_choose_eq]
have hk_fact_pos : (0 : ℝ) < k ! := by exact mod_cast factorial_pos k
exact (le_div_iff₀ hk_fact_pos).mpr h_main
```

```
theorem choose_ge_pow_div (M k : ℕ) (hk : k ≤ M) :
    ((M : ℝ) / k) ^ k ≤ (M.choose k : ℝ) := by
  first
    | exact choose_ge_pow_div_aiprover_s9
    | (intros; apply choose_ge_pow_div_aiprover_s9 <;> assumption)

theorem asymp_log_ineq_growth_logbound (δ x : ℝ) (hδ : 0 < δ) (hx : 1 ≤ x) :
    Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1)
      ≤ Real.log 11 + (2 * δ + 1) * Real.log x := by
  have hxpos : 0 < x := by linarith
  have hlog2_pos : 0 < Real.log 2 := Real.log_pos (by norm_num : (1 : ℝ) < 2)
  have hxp_pos : 0 < x ^ (2 * δ) := Real.rpow_pos_of_pos hxpos _
  have hlogx_nonneg : 0 ≤ Real.log x := Real.log_nonneg hx

  have h_log5_lt_2 : Real.log 5 < 2 := by
    have h_exp2_gt_5 : (5 : ℝ) < Real.exp 2 := by
      have h_sum_le_exp2 : (∑ m ∈ Finset.range 8, (2 : ℝ) ^ m / m.factorial) ≤ Real.exp 2 :=
        Real.sum_le_exp_of_nonneg (by norm_num) 8
      have h_sum_gt_5 : (5 : ℝ) < (∑ m ∈ Finset.range 8, (2 : ℝ) ^ m / m.factorial) := by
        have h_eq : (∑ m ∈ Finset.range 8, (2 : ℝ) ^ m / m.factorial) =
            155/21 := by
          simp [Finset.sum_range_succ]
          ring
        rw [h_eq]
        norm_num
      linarith
    have h_log_eq : Real.log (Real.exp 2) = 2 := Real.log_exp _
    have h_log_lt : Real.log 5 < Real.log (Real.exp 2) :=
      Real.log_lt_log (by norm_num : 0 < (5 : ℝ)) h_exp2_gt_5
    linarith

  have h_10log2_gt_3 : 3 < 10 * Real.log 2 := by
    have h_log2_gt_3_10 : 3/10 < Real.log 2 := by
      have h_exp_lt_2' : Real.exp (3/10) < 2 := by
        have h_sum_lt_2 : (∑ m ∈ Finset.range 8, ((3/10 : ℝ)) ^ m / m.factorial) < 2 := by
          have h_eq : (∑ m ∈ Finset.range 8, ((3/10 : ℝ)) ^ m / m.factorial) =
              1 + 3/10 + ((3/10)^2)/2 + ((3/10)^3)/6 + ((3/10)^4)/24 +
              ((3/10)^5)/120 + ((3/10)^6)/720 + ((3/10)^7)/5040 := by
            simp [Finset.sum_range_succ]
            ring
          rw [h_eq]
          norm_num
        have h_upper : Real.exp (3/10) ≤ (∑ m ∈ Finset.range 8, ((3/10 : ℝ)) ^ m / m.factorial) +
          ((3/10 : ℝ)) ^ 8 * (8 + 1) / ((8 : ℕ).factorial * 8) := by
          refine Real.exp_bound' (by norm_num) (by linarith) (by norm_num)
        have h_rem_lt : ((3/10 : ℝ)) ^ 8 * (8 + 1) / ((8 : ℕ).factorial * 8) < 1 := by
          have hpos : 0 < ((3/10 : ℝ)) ^ 8 * (8 + 1) / ((8 : ℕ).factorial * 8) := by positivity
          have hlt : ((3/10 : ℝ)) ^ 8 * (8 + 1) / ((8 : ℕ).factorial * 8) < 0.0001 := by
            nlinarith
          nlinarith
        nlinarith
      have h_log_eq' : Real.log (Real.exp (3/10)) = 3/10 := Real.log_exp _
      have h_log_lt' : Real.log (Real.exp (3/10)) < Real.log 2 :=
        Real.log_lt_log (Real.exp_pos _) h_exp_lt_2'
      linarith
    nlinarith

  have h_log5x_le_10x : Real.log (5 * x) ≤ 10 * Real.log 2 * x := by
```

```
    rw [Real.log_mul (by norm_num : (5 : ℝ) ≠ 0) hxpos.ne']
    have h_logx_le_x : Real.log x ≤ x := by
      have := Real.log_le_sub_one_of_pos hxpos
      linarith
    calc
      Real.log 5 + Real.log x ≤ 2 + Real.log x := by linarith
      _ ≤ 2 + x := by nlinarith
      _ ≤ 10 * Real.log 2 * x := by
        have h : 2 ≤ (10 * Real.log 2 - 1) * 1 := by
          nlinarith
        nlinarith

  have h_main : Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1)
      ≤ Real.log 11 + (2 * δ + 1) * Real.log x := by
    have h_split : Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1) =
        Real.log (x ^ (2 * δ)) + Real.log (Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹) := by
      have hpos_inner : 0 < Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹ := by
        have hlog5x_pos : 0 < Real.log (5 * x) := Real.log_pos (by
          have : 1 < 5 * x := by nlinarith
          exact this)
        positivity
      have h_eq : x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1 =
          x ^ (2 * δ) * (Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹) := by
        field_simp [hlog2_pos.ne']
      rw [h_eq]
      exact calc
        Real.log (x ^ (2 * δ) * (Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹))
            = Real.log (x ^ (2 * δ)) + Real.log (Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹) :=
          Real.log_mul (hxp_pos.ne') (hpos_inner.ne')
        _ = Real.log (x ^ (2 * δ)) + Real.log (Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹) := rfl
    rw [h_split]
    have h_log_xpow : Real.log (x ^ (2 * δ)) = (2 * δ) * Real.log x := by
      rw [Real.log_rpow hxpos]
    rw [h_log_xpow]




    have h_inner_bound : Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹ ≤ 11 * x := by
      have h_main_bound : Real.log (5 * x) / Real.log 2 ≤ 10 * x := by
        rw [div_le_iff₀ hlog2_pos]

        simpa [mul_comm, mul_left_comm, mul_assoc] using h_log5x_le_10x
      have h_inv_le_one : (x ^ (2 * δ))⁻¹ ≤ 1 := by
        have hxp_ge_one : 1 ≤ x ^ (2 * δ) := by
          have : 0 ≤ 2 * δ := by linarith
          calc
            1 = x ^ (0 : ℝ) := by simp
            _ ≤ x ^ (2 * δ) := Real.rpow_le_rpow_of_exponent_le hx this
        have h := (inv_le_inv₀ (by positivity) (by positivity)).2 hxp_ge_one
        simpa using h
      calc
        Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹ ≤ 10 * x + (x ^ (2 * δ))⁻¹ := by
          nlinarith
        _ ≤ 10 * x + 1 := by nlinarith
        _ ≤ 11 * x := by nlinarith
    have h_inner_pos : 0 < Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹ := by
      have hlog5x_pos : 0 < Real.log (5 * x) := Real.log_pos (by
```

```
        have : 1 < 5 * x := by nlinarith
        exact this)
      positivity
    have h_log_inner : Real.log (Real.log (5 * x) / Real.log 2 + (x ^ (2 * δ))⁻¹)
        ≤ Real.log (11 * x) := by
      exact Real.log_le_log h_inner_pos h_inner_bound
    have h_log_11x : Real.log (11 * x) = Real.log 11 + Real.log x := by
      rw [Real.log_mul (by norm_num : (11 : ℝ) ≠ 0) hxpos.ne']
    rw [h_log_11x] at h_log_inner
    nlinarith
  exact h_main

theorem asymp_log_ineq_growth_eventually (δ a b : ℝ) (hδ : 0 < δ) :
    ∃ x₀ : ℝ, 1 ≤ x₀ ∧ ∀ x : ℝ, x₀ ≤ x → a ≤ Real.log x ∧ b ≤ x ^ δ := by
  have hlog : Filter.Tendsto Real.log Filter.atTop Filter.atTop := Real.tendsto_log_atTop
  have hrpow : Filter.Tendsto (fun x : ℝ => x ^ δ) Filter.atTop Filter.atTop :=
    tendsto_rpow_atTop hδ
  have hlog_event (C : ℝ) : ∃ x₀ : ℝ, ∀ x : ℝ, x₀ ≤ x → C ≤ Real.log x :=
    (Filter.tendsto_atTop_atTop.mp hlog) C
  have hrpow_event (C : ℝ) : ∃ x₀ : ℝ, ∀ x : ℝ, x₀ ≤ x → C ≤ x ^ δ :=
    (Filter.tendsto_atTop_atTop.mp hrpow) C
  rcases hlog_event a with ⟨xlog, hlogx⟩
  rcases hrpow_event b with ⟨xrpow, hrpowx⟩
  set x₀ := max (max xlog 1) xrpow with hx₀
  have hx₀_ge_1 : 1 ≤ x₀ := by
    rw [hx₀]
    exact le_trans (le_max_right xlog 1) (le_max_left _ _)
  have hxlog_le : xlog ≤ x₀ := by
    rw [hx₀]
    exact le_trans (le_max_left xlog 1) (le_max_left _ _)
  have hxrpow_le : xrpow ≤ x₀ := by
    rw [hx₀]
    exact le_max_right _ _
  refine ⟨x₀, hx₀_ge_1, λ x hx => ?_⟩
  have hxlogx : xlog ≤ x := le_trans hxlog_le hx
  have hxrpowx : xrpow ≤ x := le_trans hxrpow_le hx
  exact ⟨hlogx x hxlogx, hrpowx x hxrpowx⟩

theorem asymp_log_ineq_growth (δ c d : ℝ) (hδ : 0 < δ) (hd : 0 < d) :
    ∃ x₀ : ℝ, 1 ≤ x₀ ∧ ∀ x : ℝ, x₀ ≤ x →
      x ^ δ * Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1) + c * x ^ (2 * δ)
        < d * x ^ (2 * δ) * Real.log x := by
  obtain ⟨x₀, hx₀1, hx₀⟩ := asymp_log_ineq_growth_eventually δ (max 1 (2 * (|c| + 1) / d))
    (2 * (Real.log 11 + 2 * δ + 1) / d) hδ
  refine ⟨x₀, hx₀1, fun x hx => ?_⟩
  obtain ⟨ha, hb⟩ := hx₀ x hx
  have hx1 : 1 ≤ x := le_trans hx₀1 hx
  have hxpos : 0 < x := by linarith
  have hu : 0 < x ^ δ := Real.rpow_pos_of_pos hxpos δ
  have hL1 : 1 ≤ Real.log x := le_trans (le_max_left _ _) ha
  have hdL : 2 * (|c| + 1) ≤ Real.log x * d := (div_le_iff₀ hd).mp (le_trans (le_max_right _ _) ha)
  have hdu : 2 * (Real.log 11 + 2 * δ + 1) ≤ x ^ δ * d := (div_le_iff₀ hd).mp hb
  have hlog11 : 0 < Real.log 11 := Real.log_pos (by norm_num)
  have e1 : 2 * (Real.log 11 + 2 * δ + 1) * Real.log x ≤ x ^ δ * d * Real.log x :=
    mul_le_mul_of_nonneg_right hdu (by linarith)
  have e2 : Real.log 11 ≤ Real.log 11 * Real.log x := by nlinarith
  have e3 : 2 * (|c| + 1) * x ^ δ ≤ Real.log x * d * x ^ δ := mul_le_mul_of_nonneg_right hdL hu.le
  have e4 : c * x ^ δ < (|c| + 1) * x ^ δ :=
    mul_lt_mul_of_pos_right (by linarith [le_abs_self c]) hu
```

```
  have key : Real.log 11 + (2 * δ + 1) * Real.log x + c * x ^ δ < d * x ^ δ * Real.log x := by
    nlinarith
  have key2 := mul_lt_mul_of_pos_left key hu
  have hlog := asymp_log_ineq_growth_logbound δ x hδ hx1
  have e5 : x ^ δ * Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1)
      ≤ x ^ δ * (Real.log 11 + (2 * δ + 1) * Real.log x) := mul_le_mul_of_nonneg_left hlog hu.le
  have hsq : x ^ (2 * δ) = x ^ δ * x ^ δ := by rw [two_mul, Real.rpow_add hxpos]
  rw [hsq] at e5 ⊢
  nlinarith [e5, key2]

theorem asymp_log_ineq_pointwise (δ ε x : ℝ) (hδ : 0 < δ) (hδε : 4 * δ < ε) (hε1 : ε < 1)
    (hx : 1 < x)
    (hgrowth : x ^ δ * Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1)
        + (1 - ε) * Real.log 5 * x ^ (2 * δ) < δ / 2 * x ^ (2 * δ) * Real.log x) :
    1 ≤ (x ^ (2 * δ) : ℝ) ∧ (x ^ (2 * δ) : ℝ) ≤ x ^ (1 - δ) ∧
      x ^ δ * Real.log ((x ^ (2 * δ) : ℝ) * Real.log (5 * x) / Real.log 2 + 1)
        + (1 - ε) * ((x ^ (2 * δ) : ℝ) * Real.log (5 * x)
        < (x ^ (2 * δ) : ℝ) * Real.log (x ^ (1 - δ) / (x ^ (2 * δ) : ℝ))
↪   := by
  have hxpos : 0 < x := by linarith
  have hx1 : 1 ≤ x := hx.le
  have hG : 0 < Real.log x := Real.log_pos hx
  have hl5 : 0 < Real.log 5 := Real.log_pos (by norm_num)
  have hl2 : 0 < Real.log 2 := Real.log_pos (by norm_num)
  have hL5 : Real.log (5 * x) = Real.log 5 + Real.log x :=
    Real.log_mul (by norm_num) hxpos.ne'
  have hL : 0 ≤ Real.log (5 * x) := by rw [hL5]; linarith
  have hy1 : 1 ≤ x ^ (2 * δ) := Real.one_le_rpow hx1 (by linarith)
  have hy0 : 0 ≤ x ^ (2 * δ) := by linarith
  have hyle : x ^ (2 * δ) ≤ x ^ (1 - δ) :=
    Real.rpow_le_rpow_of_exponent_le hx1 (by linarith)
  have hxd : 0 ≤ x ^ δ := Real.rpow_nonneg hxpos.le δ
  have hk1n : 1 ≤ x ^ (2 * δ) := Nat.le_floor (by exact_mod_cast hy1)
  have hk1 : (1:ℝ) ≤ (x ^ (2 * δ) : ℝ) := by exact_mod_cast hk1n
  have hkle : (x ^ (2 * δ) : ℝ) ≤ x ^ (2 * δ) := Nat.floor_le hy0
  have hkgt : x ^ (2 * δ) < (x ^ (2 * δ) : ℝ) + 1 := Nat.lt_floor_add_one _
  set k : ℝ := (x ^ (2 * δ) : ℝ) with hk
  have hk0 : 0 ≤ k := by linarith
  have hkpos : 0 < k := by linarith
  refine ⟨hk1, by linarith, ?_⟩
  have hc : 0 ≤ Real.log (5 * x) / Real.log 2 := div_nonneg hL hl2.le
  have hdiv : k * Real.log (5 * x) / Real.log 2
      ≤ x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 := by
    rw [mul_div_assoc, mul_div_assoc]
    exact mul_le_mul_of_nonneg_right hkle hc
  have hdiv0 : 0 ≤ k * Real.log (5 * x) / Real.log 2 := by
    rw [mul_div_assoc]
    exact mul_nonneg hk0 hc
  have hlogmono : Real.log (k * Real.log (5 * x) / Real.log 2 + 1)
      ≤ Real.log (x ^ (2 * δ) * Real.log (5 * x) / Real.log 2 + 1) :=
    Real.log_le_log (by linarith) (by linarith)
  have hA := mul_le_mul_of_nonneg_left hlogmono hxd
  have hlogk : Real.log k ≤ 2 * δ * Real.log x := by
    have := Real.log_le_log hkpos hkle
    rwa [Real.log_rpow hxpos] at this
  have hR : Real.log (x ^ (1 - δ) / k) = (1 - δ) * Real.log x - Real.log k := by
    rw [Real.log_div (Real.rpow_pos_of_pos hxpos _).ne' hkpos.ne', Real.log_rpow hxpos]
  rw [hR]
```

```
  have h1 : (1 - ε) * Real.log 5 * k ≤ (1 - ε) * Real.log 5 * x ^ (2 * δ) :=
    mul_le_mul_of_nonneg_left hkle (mul_nonneg (by linarith) hl5.le)
  have h2 : k * Real.log k ≤ k * (2 * δ * Real.log x) :=
    mul_le_mul_of_nonneg_left hlogk hk0
  have h3 : δ / 2 * x ^ (2 * δ) * Real.log x ≤ δ / 2 * (k + 1) * Real.log x :=
    mul_le_mul_of_nonneg_right
      (mul_le_mul_of_nonneg_left hkgt.le (by linarith : (0:ℝ) ≤ δ / 2)) hG.le
  have h4 : 0 ≤ (k - 1) * Real.log x * δ :=
    mul_nonneg (mul_nonneg (by linarith) hG.le) hδ.le
  have h5 : 0 ≤ (ε - 4 * δ) * k * Real.log x :=
    mul_nonneg (mul_nonneg (by linarith) hk0) hG.le
  have h6 : (1 - ε) * k * Real.log (5 * x)
      = (1 - ε) * Real.log 5 * k + (1 - ε) * k * Real.log x := by
    rw [hL5]; ring
  nlinarith [hA, hgrowth, h1, h2, h3, h4, h5, h6]

theorem asymp_log_ineq (δ ε : ℝ) (hδ : 0 < δ) (hδε : 4 * δ < ε) (hε1 : ε < 1) :
    ∃ x₀ : ℝ, 1 ≤ x₀ ∧ ∀ x : ℝ, x₀ ≤ x →
      1 ≤ (x ^ (2 * δ) : ℝ) ∧ (x ^ (2 * δ) : ℝ) ≤ x ^ (1 - δ) ∧
      x ^ δ * Real.log ((x ^ (2 * δ) : ℝ) * Real.log (5 * x) / Real.log 2 + 1)
        + (1 - ε) * (x ^ (2 * δ) : ℝ) * Real.log (5 * x)
        < (x ^ (2 * δ) : ℝ) * Real.log (x ^ (1 - δ) / (x ^ (2 * δ) : ℝ))
↪  := by
  obtain ⟨x₀, hx₀1, h⟩ := asymp_log_ineq_growth δ ((1 - ε) * Real.log 5) (δ / 2) hδ (by linarith)
  refine ⟨x₀ + 1, by linarith, fun x hx => ?_⟩
  exact asymp_log_ineq_pointwise δ ε x hδ hδε hε1 (by linarith) (h x (by linarith))

theorem asymp_main_lhs_log (x δ ε : ℝ) (k : ℕ) (hx : 1 ≤ x) (hε1 : ε < 1) :
    0 < ((Nat.log 2 (5 * x ^ k) + 1 : ℕ) : ℝ) ^ x ^ δ
        * ((5 * x ^ k : ℕ) : ℝ) ^ (1 - ε) ∧
    Real.log (((Nat.log 2 (5 * x ^ k) + 1 : ℕ) : ℝ) ^ x ^ δ
        * ((5 * x ^ k : ℕ) : ℝ) ^ (1 - ε))
      ≤ x ^ δ * Real.log ((k : ℝ) * Real.log (5 * x) / Real.log 2 + 1)
        + (1 - ε) * (k : ℝ) * Real.log (5 * x) := by
  have hx5pos : 0 < (5 : ℝ) * x := by nlinarith
  have hfloor5x_pos_nat : 0 < (⌊(5 : ℝ) * x⌋ : ℕ) := Nat.floor_pos.mpr (by nlinarith)
  have hfloor5x_pos : 0 < ((⌊(5 : ℝ) * x⌋ : ℕ) : ℝ) := by exact_mod_cast hfloor5x_pos_nat
  have hfloor5x_le : (⌊(5 : ℝ) * x⌋ : ℝ) ≤ (5 : ℝ) * x := Nat.floor_le hx5pos.le
  have h_first_pos : 0 < ((Nat.log 2 (5 * x ^ k) + 1 : ℕ) : ℝ) ^ x ^ δ :=
↪  by
    positivity
  have hpos_base_B : 0 < ((5 * x ^ k : ℕ) : ℝ) := by
    exact_mod_cast pow_pos hfloor5x_pos_nat k
  have h_second_pos : 0 < ((5 * x ^ k : ℕ) : ℝ) ^ (1 - ε) :=
    Real.rpow_pos_of_pos hpos_base_B (1 - ε)
  constructor
  · exact mul_pos h_first_pos h_second_pos
  ·
    have hpos_log2 : 0 < Real.log 2 := Real.log_pos (by norm_num : (1 : ℝ) < 2)
    have hpos_log5x : 0 < Real.log ((5 : ℝ) * x) := Real.log_pos (by nlinarith)




    have hA_pos : 0 < ((Nat.log 2 (5 * x ^ k) + 1 : ℕ) : ℝ) := by positivity
    have h_split : Real.log (((Nat.log 2 (5 * x ^ k) + 1 : ℕ) : ℝ) ^ x ^ δ
        * ((5 * x ^ k : ℕ) : ℝ) ^ (1 - ε)) =
        Real.log (((Nat.log 2 (5 * x ^ k) + 1 : ℕ) : ℝ) ^ x ^ δ)
```

```
      + Real.log (((⌊5 * x^k⌋ : ℕ) : ℝ) ^ (1 - ε)) := by
    rw [Real.log_mul (by
      have : ((Nat.log 2 (⌊5 * x^k⌋ : ℕ) : ℝ) ^ ⌊x ^ δ⌋ : ℝ) ≠ 0 := by
↪ positivity
      exact this)
      (h_second_pos.ne.symm)]
  rw [h_split]

  have h_extract : Real.log (((Nat.log 2 (⌊5 * x^k⌋ : ℕ) : ℝ) ^ ⌊x ^
↪ δ⌋ : ℝ))
      + Real.log (((⌊5 * x^k⌋ : ℕ) : ℝ) ^ (1 - ε)) =
      ((⌊x ^ δ⌋ : ℕ) : ℝ) * Real.log (((Nat.log 2 (⌊5 * x^k⌋ : ℕ) :
↪ ℝ))
      + (1 - ε) * Real.log (((⌊5 * x^k⌋ : ℕ) : ℝ)) := by
    calc
      Real.log (((Nat.log 2 (⌊5 * x^k⌋ : ℕ) : ℝ) ^ ⌊x ^ δ⌋ : ℝ))
          + Real.log (((⌊5 * x^k⌋ : ℕ) : ℝ) ^ (1 - ε))
      = Real.log (((Nat.log 2 (⌊5 * x^k⌋ : ℕ) : ℝ) ^ ((⌊x ^ δ⌋ : ℕ)
↪ : ℝ))
          + Real.log (((⌊5 * x^k⌋ : ℕ) : ℝ) ^ (1 - ε)) := by
        simp
      _ = ((⌊x ^ δ⌋ : ℕ) : ℝ) * Real.log (((Nat.log 2 (⌊5 * x^k⌋ : ℕ) : ℝ) ^ k) + 1 :
↪ ℕ) : ℝ))
          + Real.log (((⌊5 * x^k⌋ : ℕ) : ℝ) ^ (1 - ε)) := by
        rw [Real.log_rpow hA_pos ((⌊x ^ δ⌋ : ℕ) : ℝ)]
      _ = ((⌊x ^ δ⌋ : ℕ) : ℝ) * Real.log (((Nat.log 2 (⌊5 * x^k⌋ : ℕ) : ℝ) ^ k) + 1 :
↪ ℕ) : ℝ))
          + (1 - ε) * Real.log (((⌊5 * x^k⌋ : ℕ) : ℝ)) := by
        rw [Real.log_rpow hpos_base_B (1 - ε)]
      _ = ((⌊x ^ δ⌋ : ℕ) : ℝ) * Real.log (((Nat.log 2 (⌊5 * x^k⌋ : ℕ) : ℝ) ^ k) + 1 :
↪ ℕ) : ℝ))
          + (1 - ε) * Real.log (((⌊5 * x^k⌋ : ℕ) : ℝ)) := rfl
  rw [h_extract]



  have h_floor_le : ((⌊x ^ δ⌋ : ℕ) : ℝ) ≤ x ^ δ := by
    have h_nonneg : 0 ≤ x ^ δ := Real.rpow_nonneg (by linarith) δ
    exact Nat.floor_le h_nonneg
  have h_one_minus_eps_nonneg : 0 ≤ 1 - ε := by nlinarith
  have h_log_arg_nonneg : 0 ≤ Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2) + 1) :=
    Real.log_nonneg (by
      have h_nonneg_div : 0 ≤ (k : ℝ) * Real.log (5 * x) / Real.log 2 :=
        div_nonneg (mul_nonneg (Nat.cast_nonneg _) hpos_log5x.le) hpos_log2.le
      nlinarith)

  have h_log_bound : (Nat.log 2 (⌊(5 : ℝ) * x^k⌋ : ℕ) : ℝ) ≤ (k : ℝ) * Real.log ((5 : ℝ) *
↪ x) / Real.log 2 := by
    let n := ⌊(5 : ℝ) * x^k⌋
    have hn_pos : n ≠ 0 := pow_ne_zero k hfloor5x_pos_nat.ne'
    have h_pow := Nat.pow_log_le_self 2 hn_pos
    have h_pow' : ((2 : ℕ) ^ Nat.log 2 n : ℝ) ≤ (n : ℝ) := by exact_mod_cast h_pow
    have h_pow'' : ((2 : ℝ) ^ (Nat.log 2 n : ℝ)) ≤ (n : ℝ) := by
      simpa [Real.rpow_natCast] using h_pow'
    have h_log_pow : Real.log ((2 : ℝ) ^ (Nat.log 2 n : ℝ)) ≤ Real.log (n : ℝ) :=
      Real.log_le_log (by positivity) h_pow''
    rw [Real.log_rpow (by norm_num : (0 : ℝ) < 2) (Nat.log 2 n : ℝ)] at h_log_pow
    have h_log_n : Real.log (n : ℝ) = (k : ℝ) * Real.log ((⌊(5 : ℝ) * x^k⌋ : ℕ) : ℝ) := by
```

```
      simp [n, Real.log_pow]
    rw [h_log_n] at h_log_pow
    have h_div : (Nat.log 2 n : ℝ) ≤ (k : ℝ) * Real.log ((⌊(5 : ℝ) * x⌋₊ : ℕ) : ℝ) /
↪  Real.log 2 := by
      rw [le_div_iff₀ hpos_log2]
      nlinarith
    have h_log_floor_le : Real.log ((⌊(5 : ℝ) * x⌋₊ : ℕ) : ℝ) ≤ Real.log ((5 : ℝ) * x) :=
      Real.log_le_log hfloor5x_pos hfloor5x_le
    calc
      (Nat.log 2 n : ℝ) ≤ (k : ℝ) * Real.log ((⌊(5 : ℝ) * x⌋₊ : ℕ) : ℝ) / Real.log 2 :=
↪  h_div
      _ ≤ (k : ℝ) * Real.log ((5 : ℝ) * x) / Real.log 2 := by
        gcongr

  have h_bound_step3 : ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((Nat.log 2 (⌊5 * x⌋₊
↪  ^ k) + 1 : ℕ) : ℝ))
      + (1 - ε) * Real.log ((⌊5 * x⌋₊ ^ k : ℕ) : ℝ))
      ≤ ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2) + 1)
        + (1 - ε) * Real.log ((⌊5 * x⌋₊ ^ k : ℕ) : ℝ)) := by
    refine add_le_add ?_ (le_refl _)
    have h_coeff_nonneg : 0 ≤ ((⌊x ^ δ⌋₊ : ℕ) : ℝ) := Nat.cast_nonneg _
    refine mul_le_mul_of_nonneg_left ?_ h_coeff_nonneg
    refine Real.log_le_log (by positivity) ?_

    push_cast

    gcongr


  have h_log_B : Real.log ((⌊5 * x⌋₊ ^ k : ℕ) : ℝ) = (k : ℝ) * Real.log ((⌊5 *
↪  x⌋₊ : ℕ) : ℝ) := by
    simp [Real.log_pow]
  have h_log_B_bound : (1 - ε) * Real.log ((⌊5 * x⌋₊ ^ k : ℕ) : ℝ)
      ≤ (1 - ε) * ((k : ℝ) * Real.log ((5 : ℝ) * x)) := by
    rw [h_log_B]
    refine mul_le_mul_of_nonneg_left ?_ h_one_minus_eps_nonneg
    refine mul_le_mul_of_nonneg_left (Real.log_le_log hfloor5x_pos hfloor5x_le) (by exact_mod_cast
↪  Nat.zero_le k)




  have h_final : ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2)
↪  + 1)
      + (1 - ε) * ((k : ℝ) * Real.log ((5 : ℝ) * x))
      ≤ x ^ δ * Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2) + 1)
        + (1 - ε) * (k : ℝ) * Real.log (5 * x) := by
    refine add_le_add (mul_le_mul_of_nonneg_right h_floor_le h_log_arg_nonneg) ?_
    ring_nf
    rfl
  calc
    ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 : ℕ) : ℝ))
        + (1 - ε) * Real.log ((⌊5 * x⌋₊ ^ k : ℕ) : ℝ))
    ≤ ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2) + 1)
        + (1 - ε) * Real.log ((⌊5 * x⌋₊ ^ k : ℕ) : ℝ)) := h_bound_step3
    _ ≤ ((⌊x ^ δ⌋₊ : ℕ) : ℝ) * Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2) + 1)
        + (1 - ε) * ((k : ℝ) * Real.log ((5 : ℝ) * x)) := by
      refine add_le_add (le_refl _) h_log_B_bound
```

```
      _ ≤ x ^ δ * Real.log (((k : ℝ) * Real.log (5 * x) / Real.log 2) + 1)
        + (1 - ε) * (k : ℝ) * Real.log (5 * x) := h_final

theorem asymp_main_rhs_log (x δ : ℝ) (k M : ℕ) (hk1 : 1 ≤ k)
    (hkx : (k : ℝ) ≤ x ^ (1 - δ)) (hM : x ^ (1 - δ) ≤ (M : ℝ)) :
    (k : ℝ) * Real.log (x ^ (1 - δ) / (k : ℝ)) ≤ Real.log (M.choose k : ℝ) := by
  have hkpos : 0 < (k : ℝ) := by
    have := Nat.one_pos.trans_le hk1
    exact_mod_cast this
  have h_nonzero_k : (k : ℝ) ≠ 0 := by linarith
  have hk_le_M_real : (k : ℝ) ≤ (M : ℝ) := by
    linarith
  have hk_le_M : (k : ℕ) ≤ M := by
    exact_mod_cast hk_le_M_real
  have hM_gt_0 : 0 < (M : ℝ) := by
    have : 1 ≤ (M : ℝ) := by
      calc
        1 ≤ (k : ℝ) := by exact_mod_cast hk1
        _ ≤ (M : ℝ) := hk_le_M_real
    linarith
  have hx_pow_pos : 0 < x ^ (1 - δ) := lt_of_lt_of_le hkpos hkx
  have hM_div_k_ge : x ^ (1 - δ) / (k : ℝ) ≤ (M : ℝ) / (k : ℝ) :=
    div_le_div_of_nonneg_right hM (by linarith)
  have h_div_ge_one : 1 ≤ x ^ (1 - δ) / (k : ℝ) := by
    calc
      1 = (k : ℝ) / (k : ℝ) := by field_simp [h_nonzero_k]
      _ ≤ x ^ (1 - δ) / (k : ℝ) := div_le_div_of_nonneg_right hkx (by linarith)
  have h_log_ineq : Real.log (x ^ (1 - δ) / (k : ℝ)) ≤ Real.log ((M : ℝ) / (k : ℝ)) :=
    Real.log_le_log (div_pos hx_pow_pos hkpos) hM_div_k_ge
  have h_choose_pos_nat : 0 < M.choose k :=
    Nat.choose_pos hk_le_M
  have h_choose_ge_pow_nat : ((M : ℝ) / (k : ℝ)) ^ k ≤ (M.choose k : ℝ) :=
    choose_ge_pow_div M k hk_le_M
  have h_choose_pos : 0 < (M.choose k : ℝ) := by exact_mod_cast h_choose_pos_nat
  have h_div_pos : 0 < ((M : ℝ) / (k : ℝ)) := div_pos hM_gt_0 hkpos
  have h_choose_ge_pow : ((M : ℝ) / (k : ℝ)) ^ (k : ℝ) ≤ (M.choose k : ℝ) := by
    have : ((M : ℝ) / (k : ℝ)) ^ (k : ℝ) = ((M : ℝ) / (k : ℝ)) ^ k := by norm_cast
    rw [this]
    exact h_choose_ge_pow_nat
  have h_log_choose_ge : Real.log (((M : ℝ) / (k : ℝ)) ^ (k : ℝ)) ≤ Real.log (M.choose k : ℝ) :=
    Real.log_le_log (Real.rpow_pos_of_pos h_div_pos (k : ℝ)) h_choose_ge_pow
  calc
    (k : ℝ) * Real.log (x ^ (1 - δ) / (k : ℝ)) ≤ (k : ℝ) * Real.log ((M : ℝ) / (k : ℝ)) := by
      nlinarith
    _ = Real.log (((M : ℝ) / (k : ℝ)) ^ (k : ℝ)) := by
      rw [Real.log_rpow h_div_pos (k : ℝ)]
    _ ≤ Real.log (M.choose k : ℝ) := h_log_choose_ge

theorem asymp_main (δ ε : ℝ) (hδ : 0 < δ) (hδε : 4 * δ < ε) (hε1 : ε < 1) :
    ∃ x₀ : ℝ, 1 ≤ x₀ ∧ ∀ x : ℝ, x₀ ≤ x → ∀ M : ℕ, x ^ (1 - δ) ≤ (M : ℝ) →
      1 ≤ U+230Ax ^ (2 * δ)U+230BU+208A ∧ U+230Ax ^ (2 * δ)U+230BU+208A ≤ M ∧
      ((Nat.log 2 (U+230A5 * xU+230BU+208A ^ U+230Ax ^ (2 * δ)U+230BU+208A) + 1 : ℕ) : ℝ) ^ U+230Ax ^
↪   δU+230BU+208A
        * ((U+230A5 * xU+230BU+208A ^ U+230Ax ^ (2 * δ)U+230BU+208A : ℕ) : ℝ) ^ (1 - ε)
        < (M.choose U+230Ax ^ (2 * δ)U+230BU+208A : ℝ) := by
  obtain ⟨x₀, hx₀1, hx₀⟩ := asymp_log_ineq δ ε hδ hδε hε1
  refine ⟨x₀, hx₀1, fun x hx M hM => ?_⟩
  obtain ⟨hk1, hkx, hlog⟩ := hx₀ x hx
```

```
  have hk1' : 1 ≤ ⌊x ^ (2 * δ)⌋₊ := by exact_mod_cast hk1
  have hkM : ⌊x ^ (2 * δ)⌋₊ ≤ M := by exact_mod_cast hkx.trans hM
  refine ⟨hk1', hkM, ?_⟩
  obtain ⟨hpos, hL⟩ := asymp_main_lhs_log x δ ε ⌊x ^ (2 * δ)⌋₊ (by linarith) hε1
  have hR := asymp_main_rhs_log x δ ⌊x ^ (2 * δ)⌋₊ M hk1' hkx hM
  have hCpos : (0 : ℝ) < (M.choose ⌊x ^ (2 * δ)⌋₊ : ℝ) :=
    Nat.cast_pos.mpr (Nat.choose_pos hkM)
  rw [← Real.log_lt_log_iff hpos hCpos]
  linarith

theorem exists_prime_finset (hH : SmoothShiftedPrimesHypothesis) (δ η : ℝ)
    (hδ : 0 < δ) (hδ4 : δ < 1 / 4) (hη : 0 < η) :
    ∃ x₀ : ℝ, ∀ x : ℝ, x₀ ≤ x → ∃ S : Finset ℕ, x ^ (1 - η) ≤ (S.card : ℝ) ∧
      ∀ p ∈ S, p.Prime ∧ 2 * x < (p : ℝ) ∧ (p : ℝ) ≤ 5 * x ∧
        IsSmoothNat (p - 1) (x ^ δ) := by
  have hη2 : 0 < η / 2 := half_pos hη
  rcases hH δ hδ hδ4 (η / 2) hη2 with ⟨x₀, hH'⟩
  let x₁ : ℝ := max x₀ 1
  have hx₁1 : 1 ≤ x₁ := le_max_right _ _
  have hx₁0 : x₀ ≤ x₁ := le_max_left _ _
  refine ⟨x₁, λ x hx => ?_⟩
  have hx1 : 1 ≤ x := hx₁1.trans hx
  have hx0 : x₀ ≤ x := hx₁0.trans hx
  have hcard := hH' x hx0

  let A : Set ℕ := {p | p.Prime ∧ 2 * x < (p : ℝ) ∧ (p : ℝ) ≤ 5 * x ∧ IsSmoothNat (p - 1) (x ^ δ)}
  have hA_finite : A.Finite := by
    have h_bound : A ⊆ {p | (p : ℝ) ≤ 5 * x} := by
      intro p ⟨_, _, hp5, _⟩; exact hp5
    have h_finite_Iic : {p : ℕ | (p : ℝ) ≤ 5 * x}.Finite := by
      have h_eq : {p : ℕ | (p : ℝ) ≤ 5 * x} = Set.Iic (⌊5 * x⌋₊ : ℕ) := by
        ext n
        simp [Set.mem_Iic]
        constructor
        · intro h
          have hpos : 0 ≤ 5 * x := by linarith
          exact (Nat.le_floor_iff hpos).mpr h
        · intro h
          have hpos : 0 ≤ 5 * x := by linarith
          have h_n_le_floor : (n : ℝ) ≤ (⌊5 * x⌋₊ : ℝ) := by exact_mod_cast h
          have h_floor_le : (⌊5 * x⌋₊ : ℝ) ≤ 5 * x := Nat.floor_le hpos
          exact h_n_le_floor.trans h_floor_le
      rw [h_eq]
      exact Set.finite_Iic _
    exact h_finite_Iic.subset h_bound

  let S := hA_finite.toFinset
  have hS_card_nat : S.card = Nat.card A := by
    haveI : Fintype A := hA_finite.fintype
    simpa [S] using hA_finite.card_toFinset
  have hcard_nat : (Nat.card A : ℝ) ≥ x ^ (1 - η / 2) := by
    simpa [A] using hcard
  have hcard_S : (S.card : ℝ) ≥ x ^ (1 - η / 2) := by
    rw [hS_card_nat]; exact hcard_nat

  have h_exp_mono : (1 : ℝ) - η ≤ (1 : ℝ) - η / 2 := by linarith
  have h_pow_mono : x ^ (1 - η) ≤ x ^ (1 - η / 2) :=
    Real.rpow_le_rpow_of_exponent_le hx1 h_exp_mono
  have hgoal : x ^ (1 - η) ≤ (S.card : ℝ) := by
```

```
    linarith
  have hS_prop : ∀ p ∈ S, p.Prime ∧ 2 * x < (p : ℝ) ∧ (p : ℝ) ≤ 5 * x ∧ IsSmoothNat (p - 1) (x ^ δ) := by
    intro p hp
    have hpA : p ∈ A := by
      simpa [S] using hp
    rcases hpA with ⟨hp_prime, hp_low, hp_high, hp_smooth⟩
    exact ⟨hp_prime, hp_low, hp_high, hp_smooth⟩
  exact ⟨S, hgoal, hS_prop⟩

theorem good_n_from_finset (S : Finset ℕ) (x δ ε : ℝ) (k : ℕ) (hx : 1 ≤ x)
    (hε1 : ε < 1)
    (hS : ∀ p ∈ S, p.Prime ∧ 2 * x < (p : ℝ) ∧ (p : ℝ) ≤ 5 * x ∧
        IsSmoothNat (p - 1) (x ^ δ))
    (hk1 : 1 ≤ k) (hkS : k ≤ S.card)
    (hineq : ((Nat.log 2 (⌊5 * x⌋₊ ^ k) + 1 : ℕ) : ℝ) ^ ⌊x ^ δ⌋₊
        * ((⌊5 * x⌋₊ ^ k : ℕ) : ℝ) ^ (1 - ε) < (S.card.choose k : ℝ)) :
    ∃ n : ℕ, x < (n : ℝ) ∧ 0 < n ∧
      ((totientFiberCard n : ℕ) : ℝ) > (n : ℝ) ^ (1 - ε) := by
  obtain ⟨N, hN⟩ : ∃ N : ℕ, N = ⌊5 * x⌋₊ := ⟨_, rfl⟩
  rw [← hN] at hineq
  have hSp : ∀ p ∈ S, 0 < p - 1 ∧ p - 1 ≤ N ∧ IsSmoothNat (p - 1) (x ^ δ) := by
    intro p hp
    obtain ⟨hpr, h2x, h5x, hsm⟩ := hS p hp
    have h2 := hpr.two_le
    have hpN : p ≤ N := by
      rw [hN]; exact Nat.le_floor h5x
    exact ⟨by omega, by omega, hsm⟩
  obtain ⟨V, hVcard, hVmem⟩ := smooth_count (N ^ k) (x ^ δ)
  have hV : ∀ Q ⊆ S, Q.card = k → ∏ p ∈ Q, (p - 1) ∈ V := by
    intro Q hQ hQk
    have h := prod_smooth_bounded Q (x ^ δ) N (fun p hp => hSp p (hQ hp))
    rw [hQk] at h
    exact hVmem _ h.1 h.2.1 h.2.2
  obtain ⟨Q, hQS, hQk, hchoose⟩ :=
    exists_large_fiber S (fun p hp => (hS p hp).1) k hkS V hV
  have hprod := prod_smooth_bounded Q (x ^ δ) N (fun p hp => hSp p (hQS hp))
  rw [hQk] at hprod
  obtain ⟨hpos, hle, -⟩ := hprod
  refine ⟨∏ p ∈ Q, (p - 1), ?_, hpos, ?_⟩
  · have hne : Q.Nonempty := Finset.card_pos.mp (by omega)
    obtain ⟨p0, hp0⟩ := hne
    obtain ⟨hpr, h2x, -, -⟩ := hS p0 (hQS hp0)
    have h1 : p0 - 1 ≤ ∏ p ∈ Q, (p - 1) :=
      Nat.le_of_dvd hpos (Finset.dvd_prod_of_mem (fun p => p - 1) hp0)
    have h2 : (x : ℝ) < ((p0 - 1 : ℕ) : ℝ) := by
      have h1p : 1 ≤ p0 := hpr.one_lt.le
      rw [Nat.cast_sub h1p]
      push_cast
      linarith
    exact lt_of_lt_of_le h2 (Nat.cast_le.mpr h1)
  · have hVR : (V.card : ℝ) ≤ ((Nat.log 2 (N ^ k) + 1 : ℕ) : ℝ) ^ ⌊x ^ δ⌋₊ := by
      exact_mod_cast hVcard
    have hnP : (((∏ p ∈ Q, (p - 1) : ℕ) : ℝ) ^ (1 - ε) ≤ ((N ^ k : ℕ) : ℝ) ^ (1 - ε) :=
      Real.rpow_le_rpow (Nat.cast_nonneg _) (Nat.cast_le.mpr hle) (by linarith)
    have hcR : (S.card.choose k : ℝ) ≤
        (V.card : ℝ) * ((totientFiberCard (∏ p ∈ Q, (p - 1)) : ℕ) : ℝ) := by
      have h := (Nat.cast_le (α := ℝ)).mpr hchoose
      rwa [Nat.cast_mul] at h
    by_contra hcon
```

```
    push_neg at hcon
    have h1 := mul_le_mul_of_nonneg_left hcon (Nat.cast_nonneg (α := ℝ) V.card)
    have h2 := mul_le_mul_of_nonneg_left hnP (Nat.cast_nonneg (α := ℝ) V.card)
    have h3 := mul_le_mul_of_nonneg_right hVR
      (Real.rpow_nonneg (Nat.cast_nonneg (α := ℝ) (N ^ k)) (1 - ε))
    have := lt_of_le_of_lt (h1.trans (h2.trans h3)) (lt_of_lt_of_le hineq hcR)
    exact lt_irrefl _ this

theorem exists_good_n_large (hH : SmoothShiftedPrimesHypothesis) (ε : ℝ)
    (hε : 0 < ε) (hε1 : ε < 1) (C : ℝ) :
    ∃ n : ℕ, C < (n : ℝ) ∧ 0 < n ∧
      ((totientFiberCard n : ℕ) : ℝ) > (n : ℝ) ^ (1 - ε) := by
  obtain ⟨x₀, hx₀⟩ := exists_prime_finset hH (ε / 8) (ε / 8) (by linarith) (by linarith) (by linarith)
  obtain ⟨x₁, _hx₁1, hx₁⟩ := asymp_main (ε / 8) ε (by linarith) (by linarith) hε1
  obtain ⟨x, hx0, hx1, hxC, hx1'⟩ : ∃ x : ℝ, x₀ ≤ x ∧ x₁ ≤ x ∧ C ≤ x ∧ 1 ≤ x :=
    ⟨max (max x₀ x₁) (max C 1),
      le_trans (le_max_left _ _) (le_max_left _ _),
      le_trans (le_max_right _ _) (le_max_left _ _),
      le_trans (le_max_left _ _) (le_max_right _ _),
      le_trans (le_max_right _ _) (le_max_right _ _)⟩
  obtain ⟨S, hScard, hS⟩ := hx₀ x hx0
  obtain ⟨hk1, hkS, hineq⟩ := hx₁ x hx1 S.card hScard
  obtain ⟨n, hxn, hn0, hn⟩ :=
    good_n_from_finset S x (ε / 8) ε x ^ (2 * (ε / 8)) hx1' hε1 hS hk1 hkS hineq
  exact ⟨n, lt_of_le_of_lt hxC hxn, hn0, hn⟩

theorem infinite_of_lt_one (hH : SmoothShiftedPrimesHypothesis) (ε : ℝ)
    (hε : 0 < ε) (hε1 : ε < 1) :
    Set.Infinite {n : ℕ | 0 < n ∧ ((totientFiberCard n : ℕ) : ℝ) > (n : ℝ) ^ (1 - ε)} := by



  by_contra! h_fin

  have h_fin_set : Set.Finite {n : ℕ | 0 < n ∧ ((totientFiberCard n : ℕ) : ℝ) > (n : ℝ) ^ (1 - ε)} := by
    rwa [Set.not_infinite] at h_fin
  obtain ⟨M, hM⟩ := Set.Finite.bddAbove h_fin_set


  obtain ⟨n, hnC, hnpos, hn⟩ := exists_good_n_large hH ε hε hε1 (max (M : ℝ) 0)
  have hn_gt_M : (M : ℝ) < (n : ℝ) := by
    have : max (M : ℝ) 0 < (n : ℝ) := hnC
    have hM_nonneg : 0 ≤ (M : ℝ) := Nat.cast_nonneg _
    have : (M : ℝ) ≤ max (M : ℝ) (0 : ℝ) := le_max_left _ _
    linarith
  have hn_nat_gt_M : M < n := by exact_mod_cast hn_gt_M
  have hM_bound : n ≤ M := hM (Set.mem_setOf.mpr ⟨hnpos, hn⟩)
  exact hn_nat_gt_M.not_le hM_bound

theorem solution
    (hH : SmoothShiftedPrimesHypothesis) (ε : ℝ) (hε : 0 < ε) :
    Set.Infinite {n : ℕ | 0 < n ∧ ((totientFiberCard n : ℕ) : ℝ) > (n : ℝ) ^ (1 - ε)} := by
  by_cases h1 : ε < 1
  · exact infinite_of_lt_one hH ε hε h1
  · push_neg at h1
    have h := infinite_of_lt_one hH (1 / 2) (by norm_num) (by norm_num)
    apply h.mono
    intro n hn
    simp only [Set.mem_setOf_eq] at hn ⊢
```

```
    refine ⟨hn.1, lt_of_le_of_lt ?_ hn.2⟩
    apply Real.rpow_le_rpow_of_exponent_le
    · exact Nat.one_le_cast.mpr hn.1
    · linarith
```